Game Matchmaking System Design and Implementation

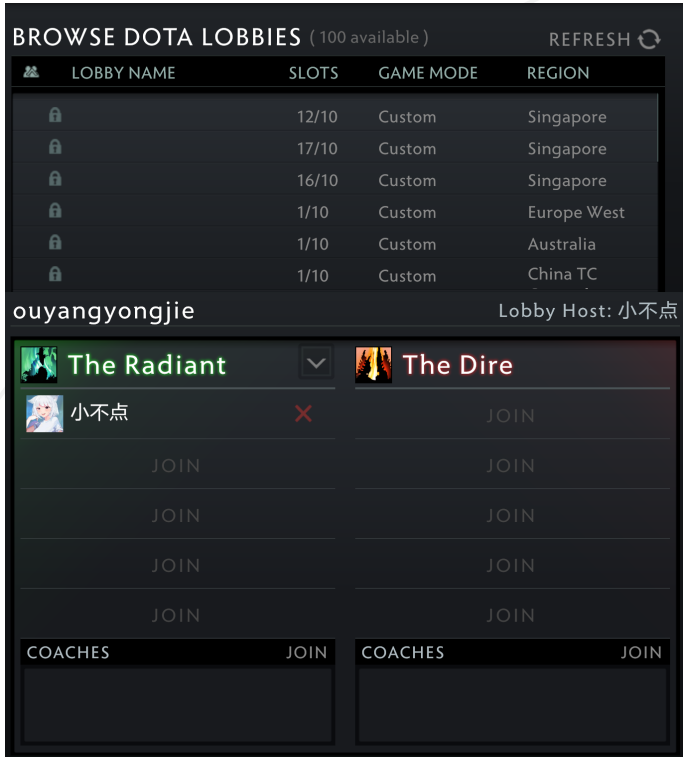
Tencent / PGOS Team
pixelcao



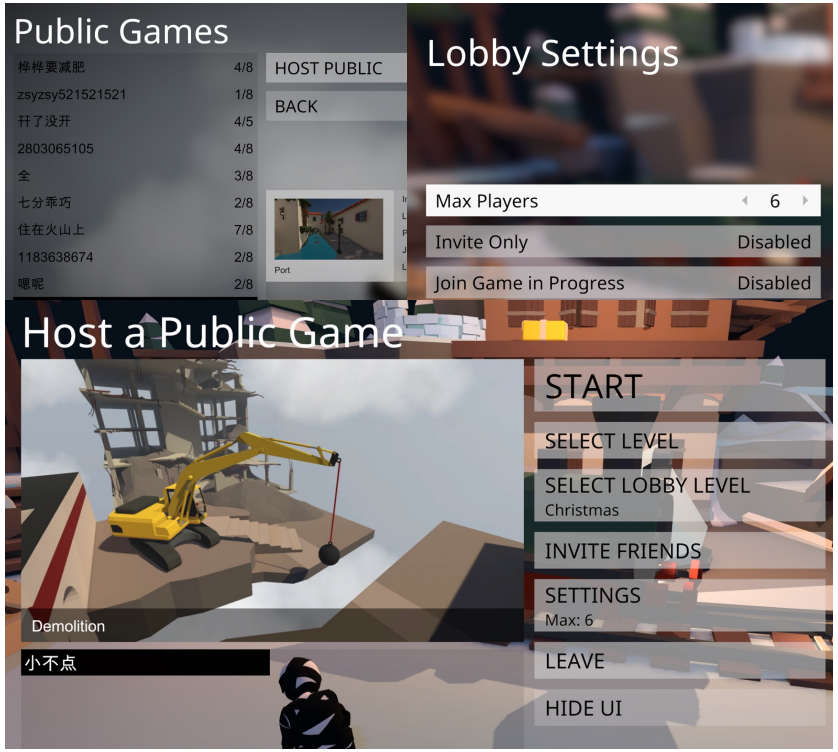
Three forms of matchmaking service

Dimension	*Lobby Matchmaker*	*World Matchmaker*	*Rule-based Matchmaker*
Match Type	*Session-Search*	*Session-Search*	*Rule-Based*
Core Mechanism	Room creation & serach	Filter-based bucket allocation	Rule-based pairing algorithm
Wait Time	Short (minutes)	Very short (seconds)	Medium (rely on config)
Rule Complexity	Basic attribute matching	Simple filtering	Complex multi-dimensional constraints
Fairness	Medium	Low	High
Use Cases	Custom rooms, teaming	Open world, lobby world	Competitive, ranked
Player Control	High (host privileges)	None	None
Scalability	Medium	Medium	High

Lobby



Dota2

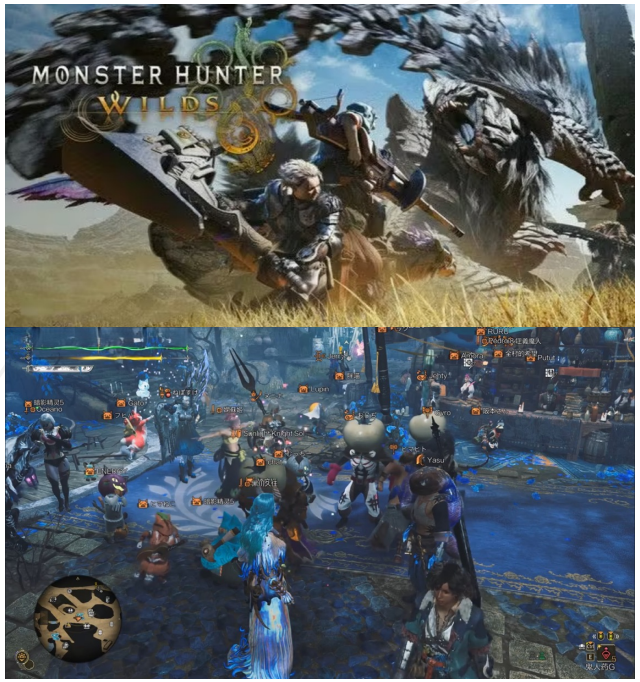


Human Fall Flat



Amont Us

World



Monster Hunter Wild



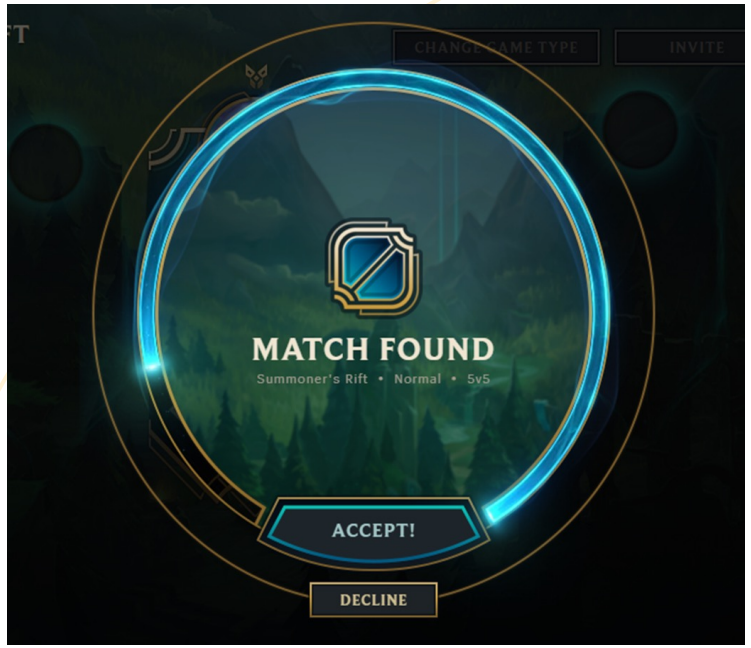
Fall Guys



Delta Force

Matchmaking

Rule-based



League of Legends



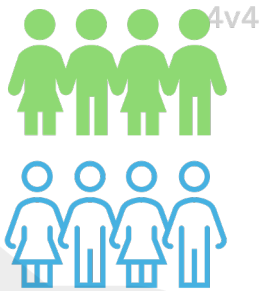
Valorant



Dota2

What matchmaking system does...

Using players in queue to generate matches that meet the requirements.



Team composition



Match fairness



Player retention

What we care about matchmaking...

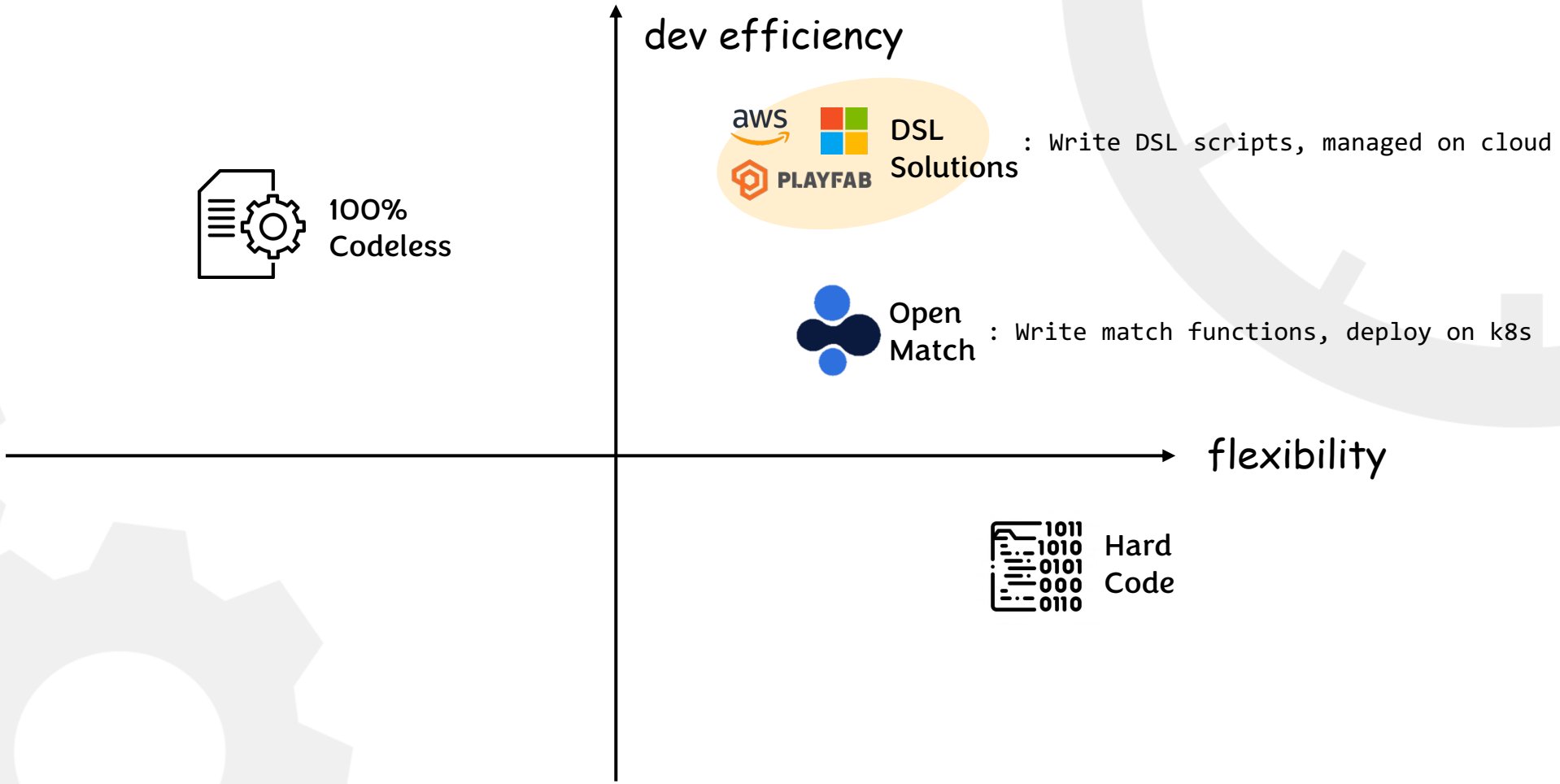


- Write less repeated code while dealing with ever-changing requirements
- Maintain matchmaking service codebase across multiple projects

- Maximize match quality and fairness
- Minimize queue time

- Aims to provide one-stop service to help both developers and players
- Deliver it as a SaaS

Approaches we considered...



Introducing “Ruleset” with an Example



Rocket League-like case:

- *Team composition 5v5, 4v4, or 3v3;*
- *Two teams must have similar skill levels;*
- *Game mode is "casual"*

Using "Ruleset" to describe matchmaking requirements

```
{
    "playerAttributes": [
        {
            "name": "mmr",
            "type": "number",
            "bitmap": false,
            "default": 1000,
            "partyAggregation": "each"
        },
        {
            "name": "mode",
            "type": "string",
            "bitmap": false,
            "default": "casual",
            "partyAggregation": "each"
        }
    ],
    "teams": [
        {
            "name": "red",
            "minPlayers": 3,
            "maxPlayers": 5,
        },
        {
            "name": "blue",
            "minPlayers": 3,
            "maxPlayers": 5,
        }
    ],
    "rules": [
        {
            "name": "fairness",
            "type": "distanceRule",
            "description": "Make the mmr of two teams close",
            "measurements": [
                "avg(teams[red].players.playerAttributes[mmr])"
            ],
            "referenceValue": "avg(teams[blue].players.playerAttributes[mmr])",
            "maxDistance": 100,
            "partyAggregation": "each"
        },
        {
            "name": "samemode",
            "type": "comparisonRule",
            "description": "Ensure players select same mode",
            "measurements": [
                "flatten(teams[*].players.playerAttributes[mode])"
            ],
            "operation": "=",
            "partyAggregation": "each"
        }
    ]
}
```

Define attributes

- ***mmr***: *stands for player skill, default to 1000*
- ***mode***: *stands for game mode, default to "casual"*

Define teams

- ***red***: *a team containing 3~5 players*
- ***blue***: *another team containing 3~5 players*

Define rules

- ***fairness***: *ensure fairness by keeping mmr close*
- ***samemode***: *ensure everyone selects "casual" mode*

Break ruleset into parts: Expression

Traditional Programming vs DSL

Traditional Approach (Imperative)

```
if (team1.avgMMR - team2.avgMMR > 100) {
    return false; // No match
}
for (player : allPlayers) {
    if (!player.mode.equals("casual")) {
        return false;
    }
}
```

DSL Approach (Declarative)

```
fairness: distanceRule {
  measurement: avg(teams[red].players.mmr)
  reference: avg(teams[blue].players.mmr)
  maxDistance: 100
}

samemode: comparisonRule {
  measurement: flatten(teams[*].players.mode)
  operation: "="
}
```

Dimension	Traditional Way	DSL Way	Description
Readability	Low	High	Intuitive business logic
Maintainability	Low	High	No recompile for rule changes
Extensibility	Low	High	Easy to add new rule types
Performance	High	Middle	Parsing overhead vs dev efficiency
Debugging	Easy	Middle	Requires dedicated tools

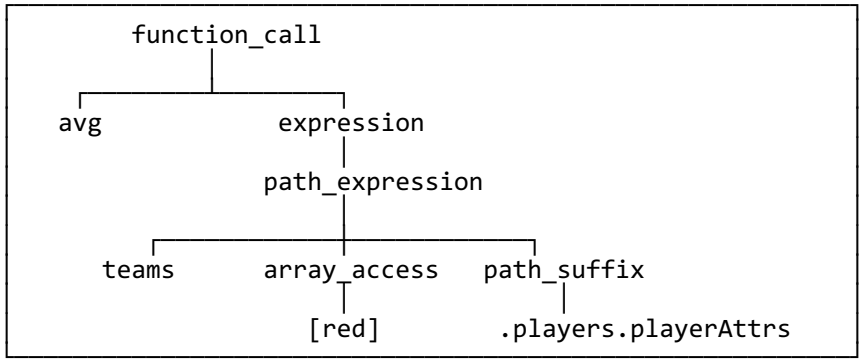
Break ruleset into parts: Expression

Input: avg(teams[red].players.playerAttributes[mmr])

Step 1: Shift-Reduce Analysis

Stack	Input	Action
$	avg(teams[red]...	shift
$ avg	(teams[red]...	shift
$ avg (	teams[red]...	shift
$ avg (teams	[red]...	shift
$ avg (teams [	red]...	shift

Step 2: Syntax Tree Construction



Step 3: Semantic Action Execution

```
1. create_function_node("avg", path_expr)
2. create_path_node("teams", array_access, suffix)
3. create_array_access_node("red")
4. create_field_access_chain(["players", "playerAttrs"])
5. Return complete AST root node
```

Stage	Brief	Implementation
Lexer	avg(teams[red].players.playerAttributes[mmr])	mmparser/lexer.go scans and generates a TOKEN stream: ACTION("avg") + LC + FIELD("teams") + LS + FIELD("red") + RS + DOT + FIELD("players")...
Parser	TOKEN stream → AST struct	mmparser/yacc.y performs shift-reduce parsing to produce: [{"avg": [{"teams": "red"}, {"players": ""}, {"playerAttributes": "mmr"}]}]
Expression Cache	AST struct cache	ruleset/expression.go caches parsed ASTs in expCache to avoid repeated parsing of the same expression.
Recursive Evaluation	AST → actual data	EvaluateExpression() identifies the "avg" action and calls retrieveTeams() to extract red team player MMR data from the proposal.
Action Execution	Aggregation functions	avg() function calculates the average of the extracted MMR array and returns the final result.
Result Returning	Calculate numbers	The result is returned in the format []{1200.0} for use by the rule engine.

Break ruleset into parts: Rule

Rule Type	Use Case	Implementation	Examples
distanceRule	Numeric range control	Calculate absolute diff between measurement and reference, check if within maxDistance	MMR gap, skill balance, exp matching, gear score balance
comparisonRule	Exact matching	Consistency check on measurement array, supports =, !=, >, < operators	Game mode, region, language, team size equality, platform type
latencyRule	Network latency control	Measure player-to-server latency, ensure within acceptable range	Server selection, low latency
collectionRule	Set quantity constraints	Count elements meeting conditions, check minCount/maxCount	Team role ratios, skill distribution, gear type counts, exp tier distribution

Topic List



- How to Build Fair Matches
- What If DSL Is Difficult to Debug
- How to Balance Match Quality and Queue Time
- Solving Performance Bottlenecks with Data Structures
- Improving Performance Limits Through Deployment Strategies
- Real Case: When Clients Want Multi-Mode Matchmaking
- Emerging Trend: Engagement Oriented, Not Skill Based
- How to Extend MMR Calculation Mechanism with Virtual Server
- Consider Looter Shooters and Battle Royales

Team size mismatch

```
{
    "playerAttributes": [
        {
            "name": "mmr",
            "type": "number",
            "bitmap": false,
            "default": 1000,
            "partyAggregation": "each"
        },
        {
            "name": "mode",
            "type": "string",
            "bitmap": false,
            "default": "casual",
            "partyAggregation": "each"
        }
    ],
    "teams": [
        {
            "name": "red",
            "minPlayers": 3,
            "maxPlayers": 5,
        },
        {
            "name": "blue",
            "minPlayers": 3,
            "maxPlayers": 5,
        }
    ],
    "rules": [
        {
            "name": "fairness",
            "type": "distanceRule",
            "description": "Make the mmr of two teams close",
            "measurements": [
                "avg(teams[red].players.playerAttributes[mmr])"
            ],
            "referenceValue": "avg(teams[blue].players.playerAttributes[mmr])",
            "maxDistance": 100,
            "partyAggregation": "each"
        },
        {
            "name": "samemode",
            "type": "comparisonRule",
            "description": "Ensure players select same mode",
            "measurements": [
                "flatten(teams[*].players.playerAttributes[mode])"
            ],
            "operation": "=",
            "partyAggregation": "each"
        }
    ]
}
```

PlayerID	MMR	GameMode	Assigned Team
P001	1180	casual	red
P002	1220	casual	red
P003	1150	casual	red
P004	1200	casual	blue
P005	1190	casual	blue
P006	1210	casual	blue
P007	1160	casual	blue

3

4



Legal but not appropriate :(

Team size mismatch

```
{
        "name": "equalteamsize",
        "type": "comparisonRule",
        "description": "Ensure two team size are equal",
        "measurements": [
            "count(teams[red].players)"
        ],
        "referenceValue": "count(teams[blue].players)",
        "operation": "=",
        "partyAggregation": "each"
}
```

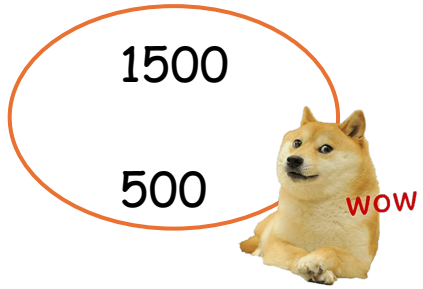
PlayerID	MMR	GameMode	Assigned Team	
P001	1180	casual	red	3
P002	1220	casual	red	
P003	1150	casual	red	
P004	1200	casual	blue	3
P005	1190	casual	blue	
P006	1210	casual	blue	
~~P007~~	~~1160~~	~~casual~~	~~blue~~	

Now that's fair :)

Skill gap too wide

```
{
    "playerAttributes": [
        {
            "name": "mmr",
            "type": "number",
            "bitmap": false,
            "default": 1000,
            "partyAggregation": "each"
        },
        {
            "name": "mode",
            "type": "string",
            "bitmap": false,
            "default": "casual",
            "partyAggregation": "each"
        }
    ],
    "teams": [
        {
            "name": "red",
            "minPlayers": 3,
            "maxPlayers": 5,
        },
        {
            "name": "blue",
            "minPlayers": 3,
            "maxPlayers": 5,
        }
    ],
    "rules": [
        {
            "name": "fairness",
            "type": "distanceRule",
            "description": "Make the mmr of two teams close",
            "measurements": [
                "avg(teams[red].players.playerAttributes[mmr])"
            ],
            "referenceValue": "avg(teams[blue].players.playerAttributes[mmr])",
            "maxDistance": 100,
            "partyAggregation": "each"
        },
        {
            "name": "samemode",
            "type": "comparisonRule",
            "description": "Ensure players select same mode",
            "measurements": [
                "flatten(teams[*].players.playerAttributes[mode])"
            ],
            "operation": "=",
            "partyAggregation": "each"
        }
    ]
}
```

PlayerID	MMR	GameMode	Assigned Team
P001	1500	casual	red
P002	500	casual	red
P003	1000	casual	red
P004	1000	casual	blue
P005	950	casual	blue
P006	1050	casual	blue



Legal but not appropriate :(

Skill gap too wide

```
{
        "name": "skillgap",
        "type": "distanceRule",
        "description": "Limit the skill gap of players",
        "measurements": [
            "max(flatten(teams[red].players.playerAttributes[mmr]))"
        ],
        "referenceValue": "min(flatten(teams[red].players.playerAttributes[mmr]))",
        "maxDistance": 200,
        "partyAggregation": "each"
}
```

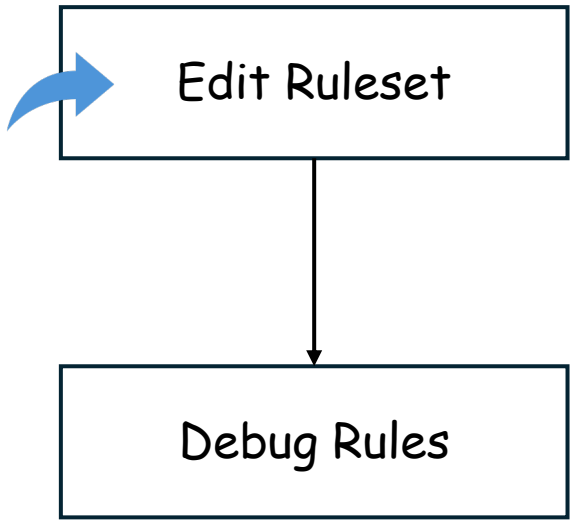
PlayerID	MMR	GameMode	Assigned Team
P001	1100	casual	red
P002	900	casual	red
P003	1000	casual	red
P004	1000	casual	blue
P005	950	casual	blue
P006	1050	casual	blue

Now that's fair :)

Topic List

- How to Build Fair Matches
-  What If DSL Is Difficult to Debug
- How to Balance Match Quality and Queue Time
- Solving Performance Bottlenecks with Data Structures
- Improving Performance Limits Through Deployment Strategies
- Real Case: When Clients Want Multi-Mode Matchmaking
- Emerging Trend: Engagement Oriented, Not Skill Based
- How to Extend MMR Calculation Mechanism with Virtual Server
- Consider Looter Shooters and Battle Royales

How to build a ruleset





Visual Editor

Player Attributes

Data Source	Key	Name	Type	Default Value	Bitmap	Party Aggregation	Operation
Client Request	-	mmr	Number	1000	No	Each	Edit Delete
Client Request	-	mode	String	casual	-	Each	Edit Delete

+ Add Attribute

Teams

Name	Min Players	Max Players	Min Quantity	Max Quantity	Operation
red	3	5	1	1	Edit Delete
blue	3	5	1	1	Edit Delete

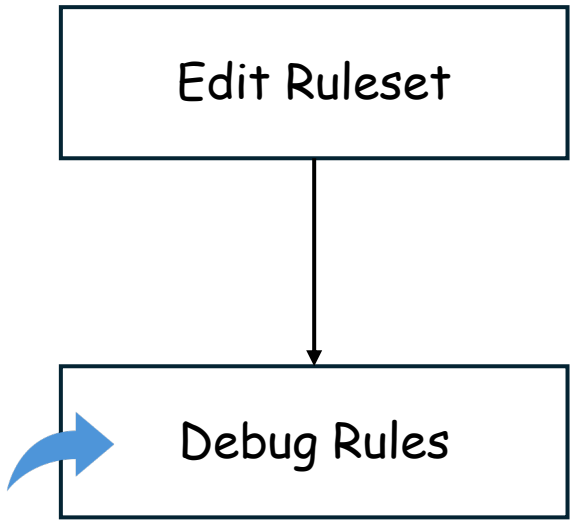
+ Add Team

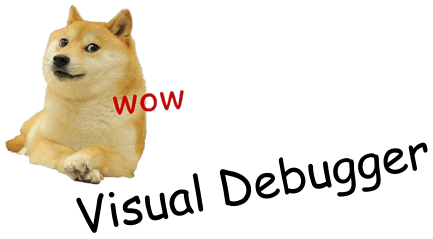
Rules

Name	Type	Description	Detail	Operation
fairness	Distance Rule	Make the mmr of ...	Measurements: ["avg(teams[red].players.playerAttributes[mmr])"] Reference Value: avg(teams[blue].players.playerAttributes[mmr]) Min Distance: Max Distance: 100 Party Aggregation: Each	Edit Delete
samemode	Comparison Rule	Ensure players sel...	Measurements: ["flatten(teams[*].players.playerAttributes[mode])"] Reference Value: Operation: = Party Aggregation: Each	Edit Delete
equalteamsize	Comparison Rule	Ensure two team ...	Measurements: ["count(teams[red].players)"] Reference Value: count(teams[blue].players) Operation: = Party Aggregation: Each	Edit Delete
skillgap	Distance Rule	Limit the skill gap ...	Measurements: ["max(flatten(teams[red].players.playerAttributes[mmr]))"] Reference Value: min(flatten(teams[red].players.playerAttributes[mmr])) Min Distance: Max Distance: 100 Party Aggregation: Each	Edit Delete

+ Add Rule

How to build a ruleset





Visual Debugger

Fake Matched Players

Team	Player ID	Attribute (mmr)	Attribute (mode)	Operation
red	player-1	1231.949166	casual	Edit
red	player-2	824.890609	casual	Edit
red	player-3	971.804810	ranked	Edit
blue	player-6	965.704134	ranked	Edit
blue	player-7	922.935254	casual	Edit
blue	player-8	987.699488	casual	Edit

Remove Selected Row(s) + Add a Fake Matched Player Quick Import from Fake Player Dataset

Execute Rules with Fake Matched Players

Rule Name	Rule Type	Measurements Expression	Reference Expression	Execution Result
fairness	Distance Rule	avg(teams[red].players.playerAttributes[mmr]) -> [1009.5481950000002]	avg(teams[blue].players.playerAttributes[mmr]) -> [958.7796253333332]	Pass
samemode	Comparison Rule	flatten(teams[*].players.playerAttributes[mode]) -> [ranked casual casual casual casual ranked]		Not Pass
equalteamsize	Comparison Rule	count(teams[red].players) -> [3]	count(teams[blue].players) -> 3	Pass
skillgap	Distance Rule	max(flatten(teams[red].players.playerAttributes[mmr])) -> [1231.949166]	min(flatten(teams[red].players.playerAttributes[mmr])) -> [824.890609]	Not Pass

✓ Execute Rules

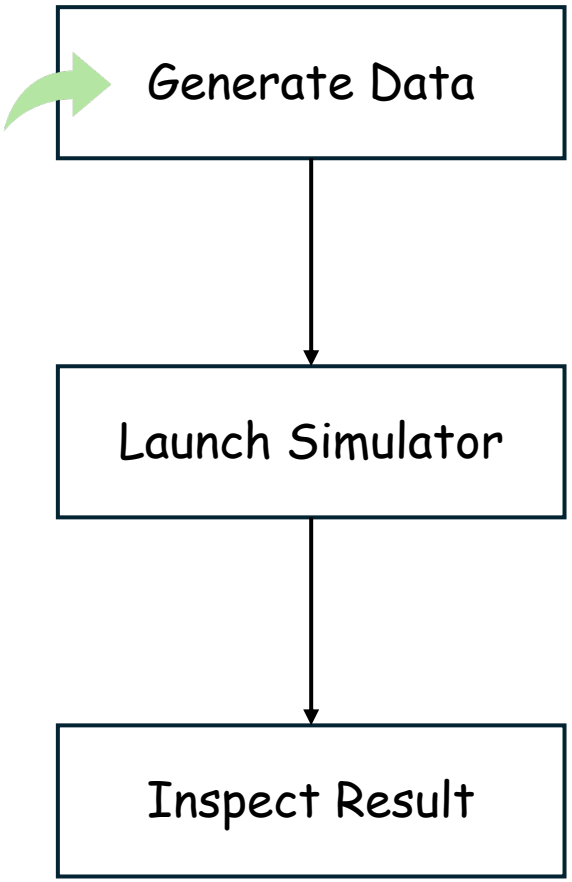
Evaluate Expression Based on Fake Matched Players

1. Expression: flatten(teams[red].players.playerAttributes[mmr]) Result: [1231.949166 824.890609 971.80481]

+ Add a Expression

✓ Evaluate Expressions

How to test a ruleset



* Name: rocketleague-sample 19/128

* Ticket Scale: 200

Multi-player Matchmaking Tickets Configuration

Ticket Configuration

Ticket Player Count	Weight	Operation
1	100	Delete

+ Add Ticket Configuration

Player Attribute Data Characteristics

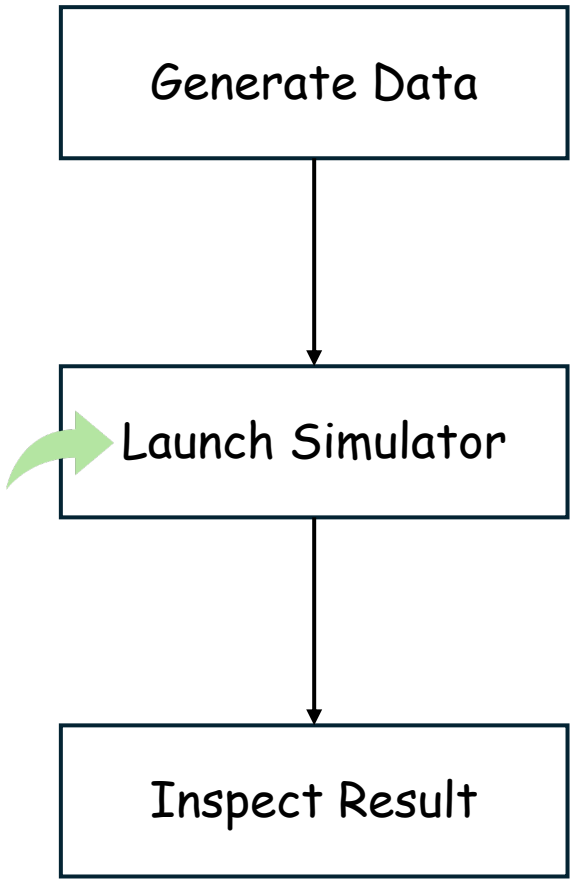
Add attributes for players in the package, configure the stats for the attribute type and values here. These properties will be used during matchmaking tests and ruleset tests.

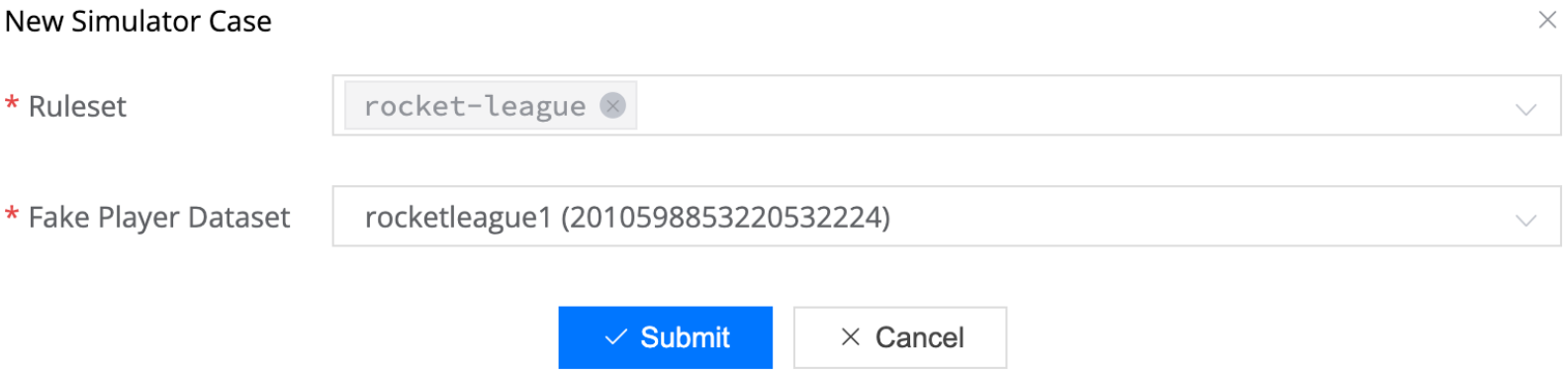
Quick import from Ruleset | + Add Attribute

Attribute Name	Type	Distribution Type	Normal Distribution Params	Operation
mmr	Number	Normal	Integer: No Expectations: 1000, Variance: 200 Left Border: 700, Right Border: 1300	Edit Delete
mode	String	-	casual: 100 ranked: 100	Edit Delete

× Cancel | ✓ Create

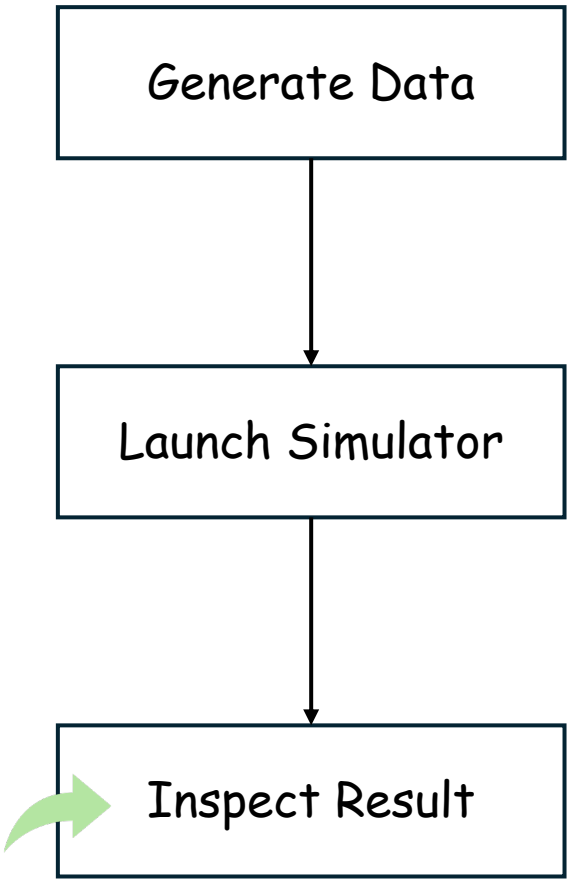
How to test a ruleset

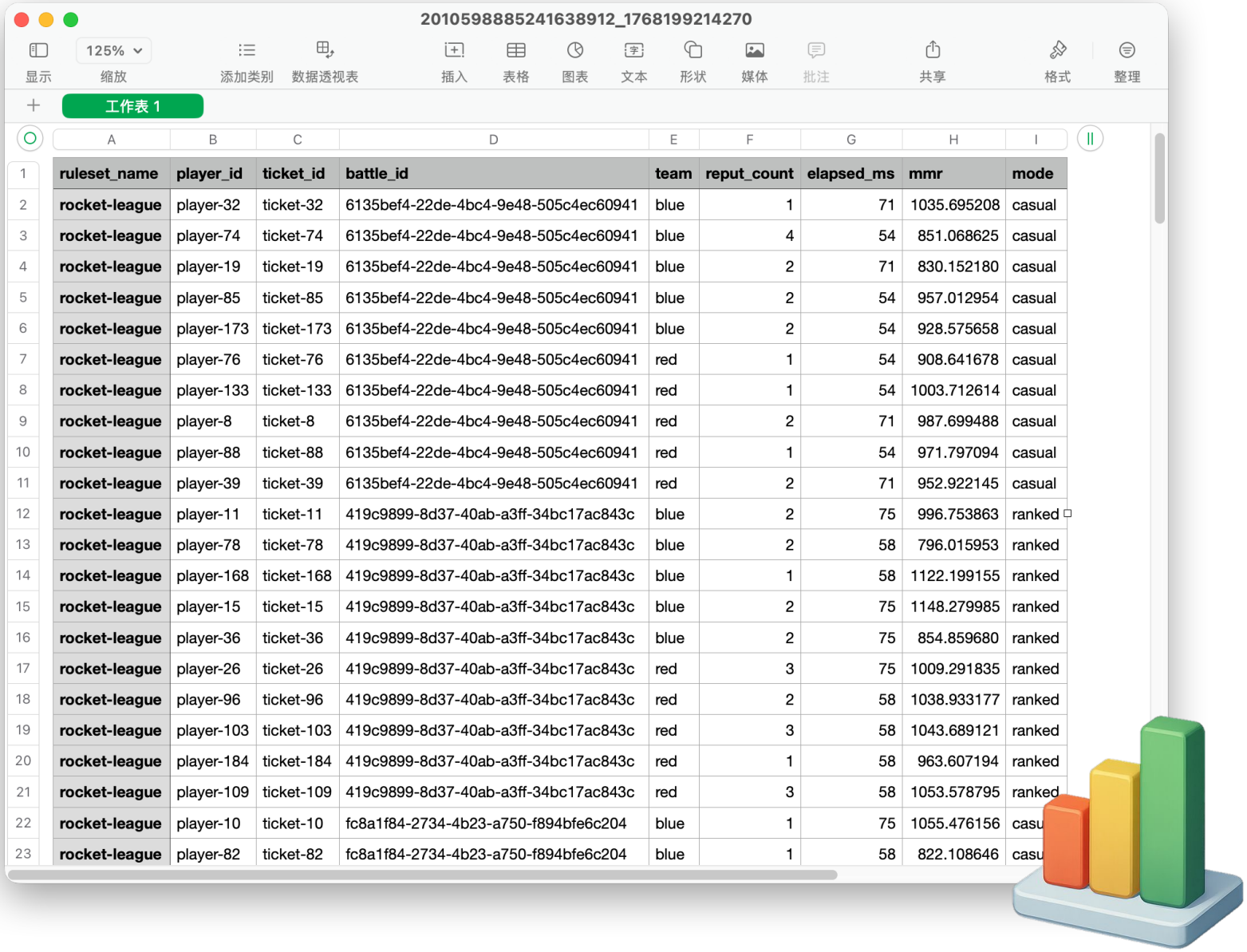




A Simulator is a single container that houses matchmaking rules, a ticket pool, and matching algorithms, which is isolated from the online service.

How to test a ruleset





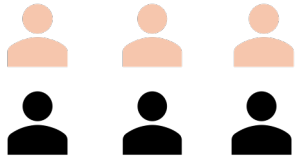
	A	B	C	D	E	F	G	H	I
1	ruleset_name	player_id	ticket_id	battle_id	team	reput_count	elapsed_ms	mmr	mode
2	rocket-league	player-32	ticket-32	6135bef4-22de-4bc4-9e48-505c4ec60941	blue	1	71	1035.695208	casual
3	rocket-league	player-74	ticket-74	6135bef4-22de-4bc4-9e48-505c4ec60941	blue	4	54	851.068625	casual
4	rocket-league	player-19	ticket-19	6135bef4-22de-4bc4-9e48-505c4ec60941	blue	2	71	830.152180	casual
5	rocket-league	player-85	ticket-85	6135bef4-22de-4bc4-9e48-505c4ec60941	blue	2	54	957.012954	casual
6	rocket-league	player-173	ticket-173	6135bef4-22de-4bc4-9e48-505c4ec60941	blue	2	54	928.575658	casual
7	rocket-league	player-76	ticket-76	6135bef4-22de-4bc4-9e48-505c4ec60941	red	1	54	908.641678	casual
8	rocket-league	player-133	ticket-133	6135bef4-22de-4bc4-9e48-505c4ec60941	red	1	54	1003.712614	casual
9	rocket-league	player-8	ticket-8	6135bef4-22de-4bc4-9e48-505c4ec60941	red	2	71	987.699488	casual
10	rocket-league	player-88	ticket-88	6135bef4-22de-4bc4-9e48-505c4ec60941	red	1	54	971.797094	casual
11	rocket-league	player-39	ticket-39	6135bef4-22de-4bc4-9e48-505c4ec60941	red	2	71	952.922145	casual
12	rocket-league	player-11	ticket-11	419c9899-8d37-40ab-a3ff-34bc17ac843c	blue	2	75	996.753863	ranked
13	rocket-league	player-78	ticket-78	419c9899-8d37-40ab-a3ff-34bc17ac843c	blue	2	58	796.015953	ranked
14	rocket-league	player-168	ticket-168	419c9899-8d37-40ab-a3ff-34bc17ac843c	blue	1	58	1122.199155	ranked
15	rocket-league	player-15	ticket-15	419c9899-8d37-40ab-a3ff-34bc17ac843c	blue	2	75	1148.279985	ranked
16	rocket-league	player-36	ticket-36	419c9899-8d37-40ab-a3ff-34bc17ac843c	blue	2	75	854.859680	ranked
17	rocket-league	player-26	ticket-26	419c9899-8d37-40ab-a3ff-34bc17ac843c	red	3	75	1009.291835	ranked
18	rocket-league	player-96	ticket-96	419c9899-8d37-40ab-a3ff-34bc17ac843c	red	2	58	1038.933177	ranked
19	rocket-league	player-103	ticket-103	419c9899-8d37-40ab-a3ff-34bc17ac843c	red	3	58	1043.689121	ranked
20	rocket-league	player-184	ticket-184	419c9899-8d37-40ab-a3ff-34bc17ac843c	red	1	58	963.607194	ranked
21	rocket-league	player-109	ticket-109	419c9899-8d37-40ab-a3ff-34bc17ac843c	red	3	58	1053.578795	ranked
22	rocket-league	player-10	ticket-10	fc8a1f84-2734-4b23-a750-f894bfe6c204	blue	1	75	1055.476156	casu
23	rocket-league	player-82	ticket-82	fc8a1f84-2734-4b23-a750-f894bfe6c204	blue	1	58	822.108646	casu

Topic List

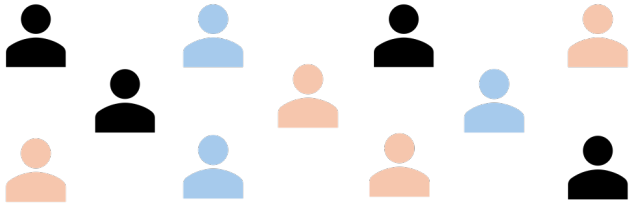
- How to Build Fair Matches
- What If DSL Is Difficult to Debug
-  How to Balance Match Quality and Queue Time
- Solving Performance Bottlenecks with Data Structures
- Improving Performance Limits Through Deployment Strategies
- Real Case: When Clients Want Multi-Mode Matchmaking
- Emerging Trend: Engagement Oriented, Not Skill Based
- How to Extend MMR Calculation Mechanism with Virtual Server
- Consider Looter Shooters and Battle Royales

Skill Balance vs Queue Time

match quality (skill balance)



[High quality match needs long wait time]



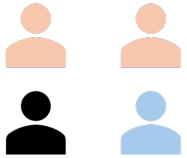
[Require massive concurrent online players]

low queue time

[Badly written rulesets]



[Typically too low player count]



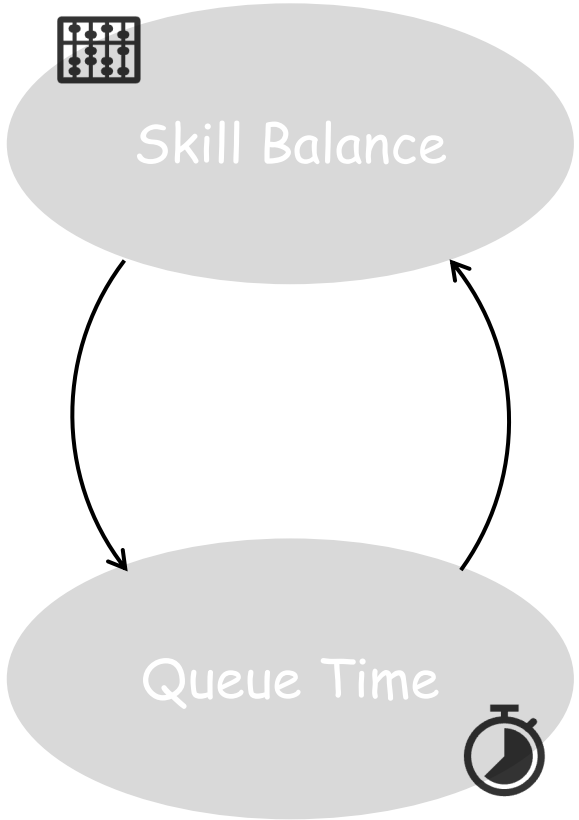
Skill Balance vs Queue Time

```
{
    "playerAttributes": [
        {
            "name": "mmr",
            "type": "number",
            "bitmap": false,
            "default": 1000,
            "partyAggregation": "each"
        },
        {
            "name": "mode",
            "type": "string",
            "bitmap": false,
            "default": "casual",
            "partyAggregation": "each"
        }
    ],
    "teams": [
        {
            "name": "red",
            "minPlayers": 3,
            "maxPlayers": 5,
        },
        {
            "name": "blue",
            "minPlayers": 3,
            "maxPlayers": 5,
        }
    ],
    "rules": [
        {
            "name": "fairness",
            "type": "distanceRule",
            "description": "Make the mmr of two teams close",
            "measurements": [
                "avg(teams[red].players.playerAttributes[mmr])"
            ],
            "referenceValue": "avg(teams[blue].players.playerAttributes[mmr])",
            "maxDistance": 100,
            "partyAggregation": "each"
        },
        {
            "name": "samemode",
            "type": "comparisonRule",
            "description": "Ensure players select same mode",
            "measurements": [
                "flatten(teams[*].players.playerAttributes[mode])"
            ],
            "operation": "=",
            "partyAggregation": "each"
        }
    ]
}
```



relax restrictions

```
"expansions": [
    {
        "target": "rules[fairness].maxDistance",
        "steps": [
            {
                "waitTimeSeconds": 15,
                "value": "200"
            },
            {
                "waitTimeSeconds": 30,
                "value": "300"
            },
            {
                "waitTimeSeconds": 60,
                "value": "500"
            }
        ]
    },
    {
        "target": "rules[skillgap].maxDistance",
        "steps": [
            {
                "waitTimeSeconds": 15,
                "value": "200"
            },
            {
                "waitTimeSeconds": 30,
                "value": "250"
            },
            {
                "waitTimeSeconds": 60,
                "value": "300"
            }
        ]
    }
```



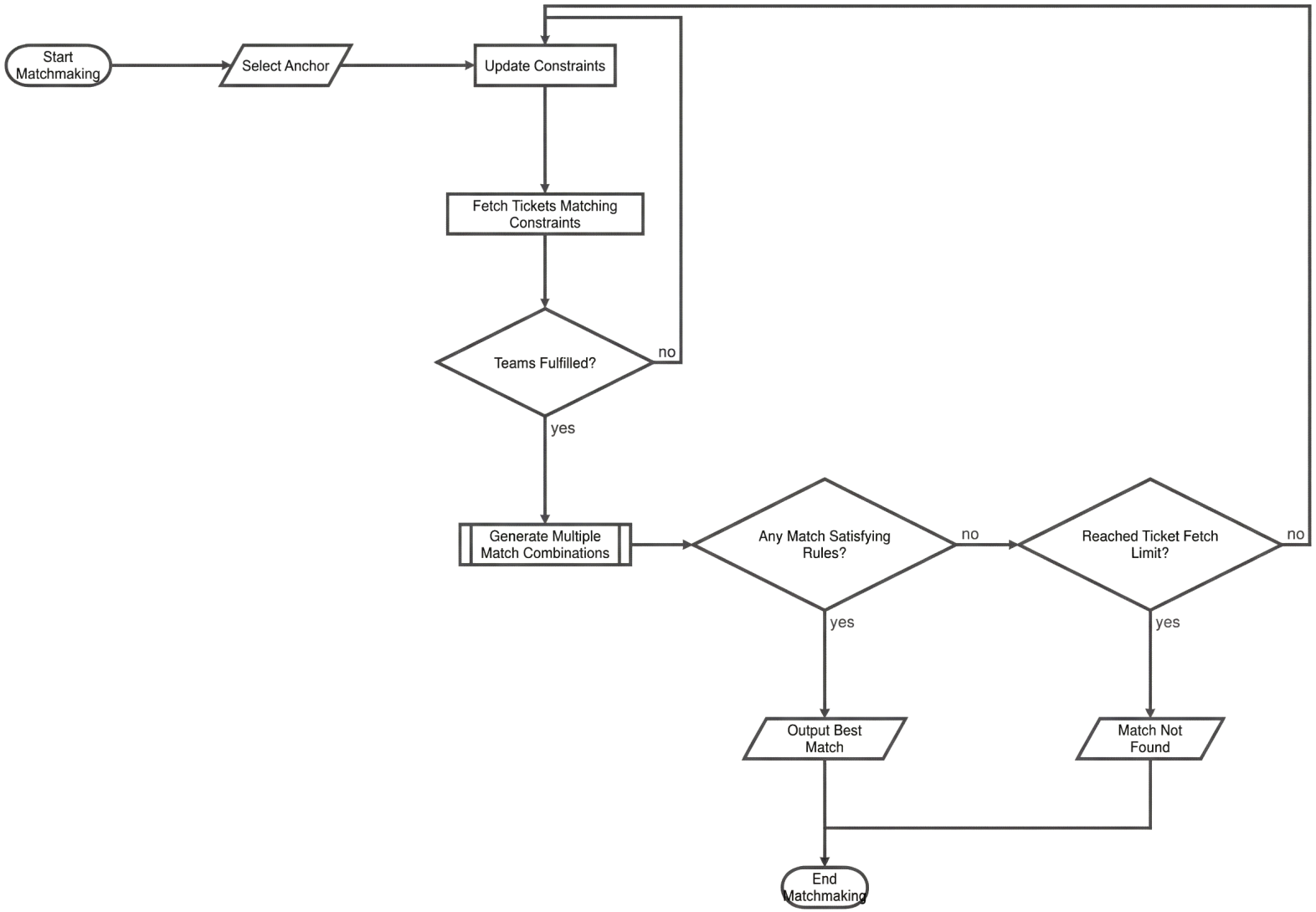
Topic List

- How to Build Fair Matches
- What If DSL Is Difficult to Debug
- How to Balance Match Quality and Queue Time
-  Solving Performance Bottlenecks with Data Structures
- Improving Performance Limits Through Deployment Strategies
- Real Case: When Clients Want Multi-Mode Matchmaking
- Emerging Trend: Engagement Oriented, Not Skill Based
- How to Extend MMR Calculation Mechanism with Virtual Server
- Consider Looter Shooters and Battle Royales

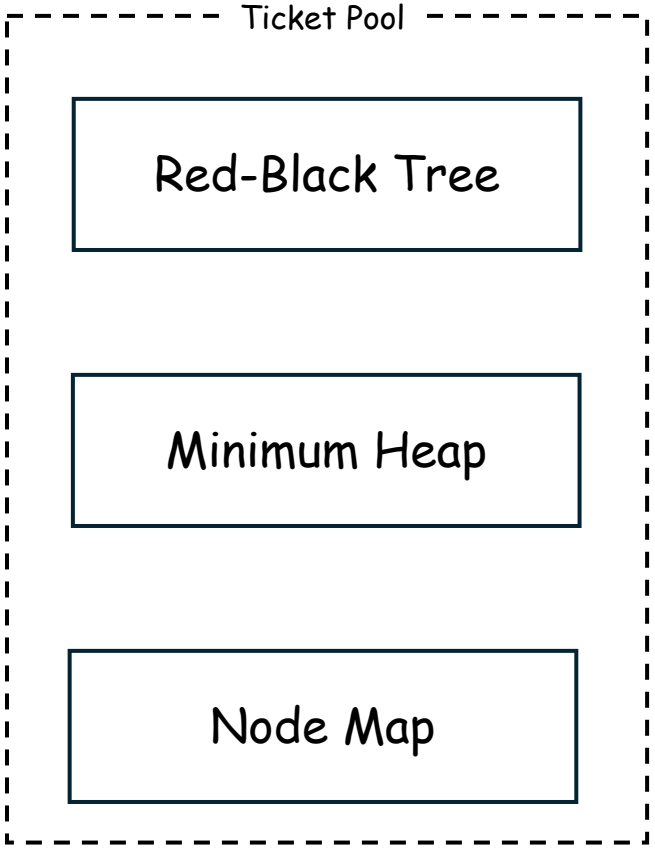
Matching Algorithm Workflow



Linear Programming



Ticket Pool Implementation



1. Sort tickets by attributes
2. Query by value range
3. O(log n) complexity

1. Used to retrieve anchor ticket
2. Prioritized queue
3. Ensure fairness of every ticket

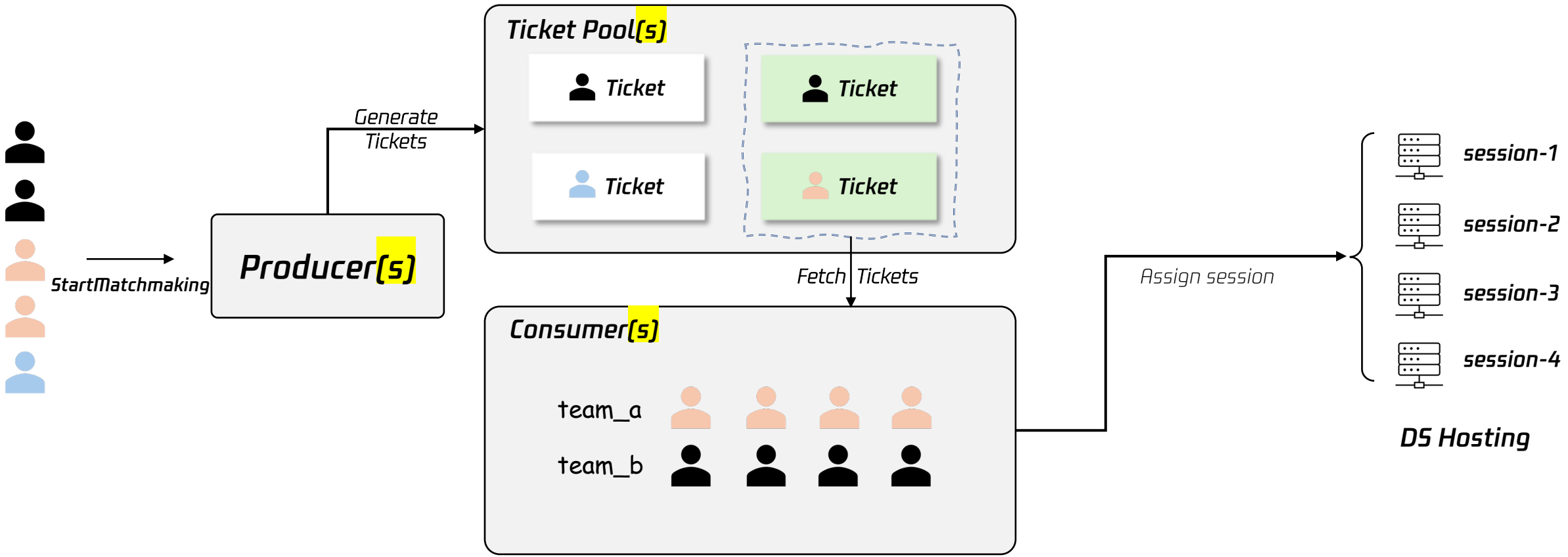
1. ID -> Ticket full info
2. TTL cleaning

Topic List

- How to Build Fair Matches
- What If DSL Is Difficult to Debug
- How to Balance Match Quality and Queue Time
- Solving Performance Bottlenecks with Data Structures

- Improving Performance Limits Through Deployment Strategies
- Real Case: When Clients Want Multi-Mode Matchmaking
- Emerging Trend: Engagement Oriented, Not Skill Based
- How to Extend MMR Calculation Mechanism with Virtual Server
- Consider Looter Shooters and Battle Royales

Matchmaking System Architecture



Attribute Filter

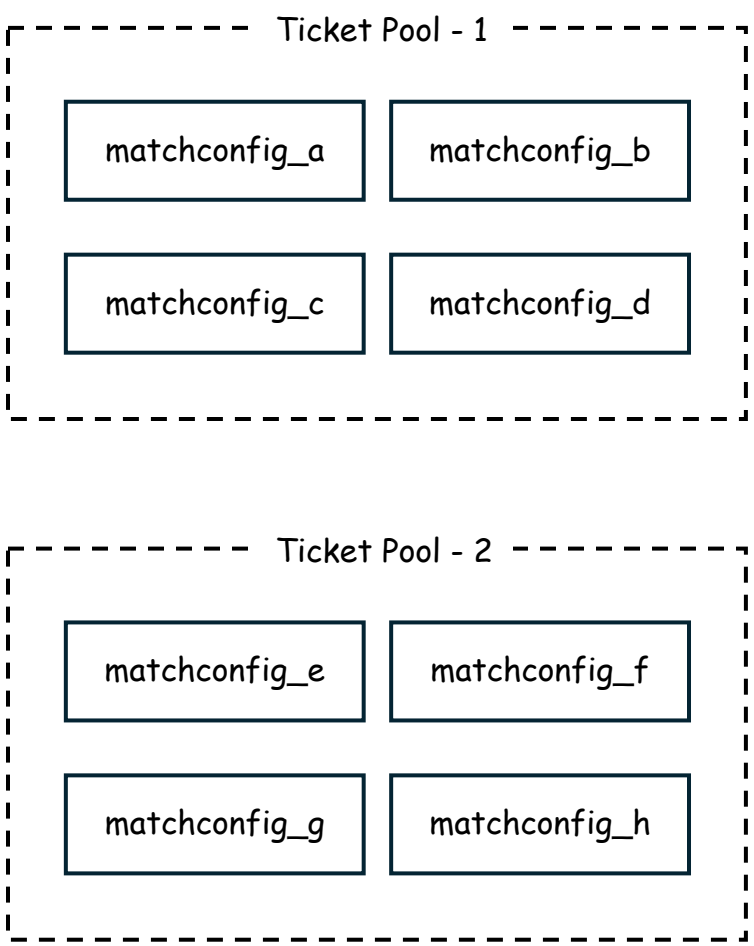
Flexible Ruleset

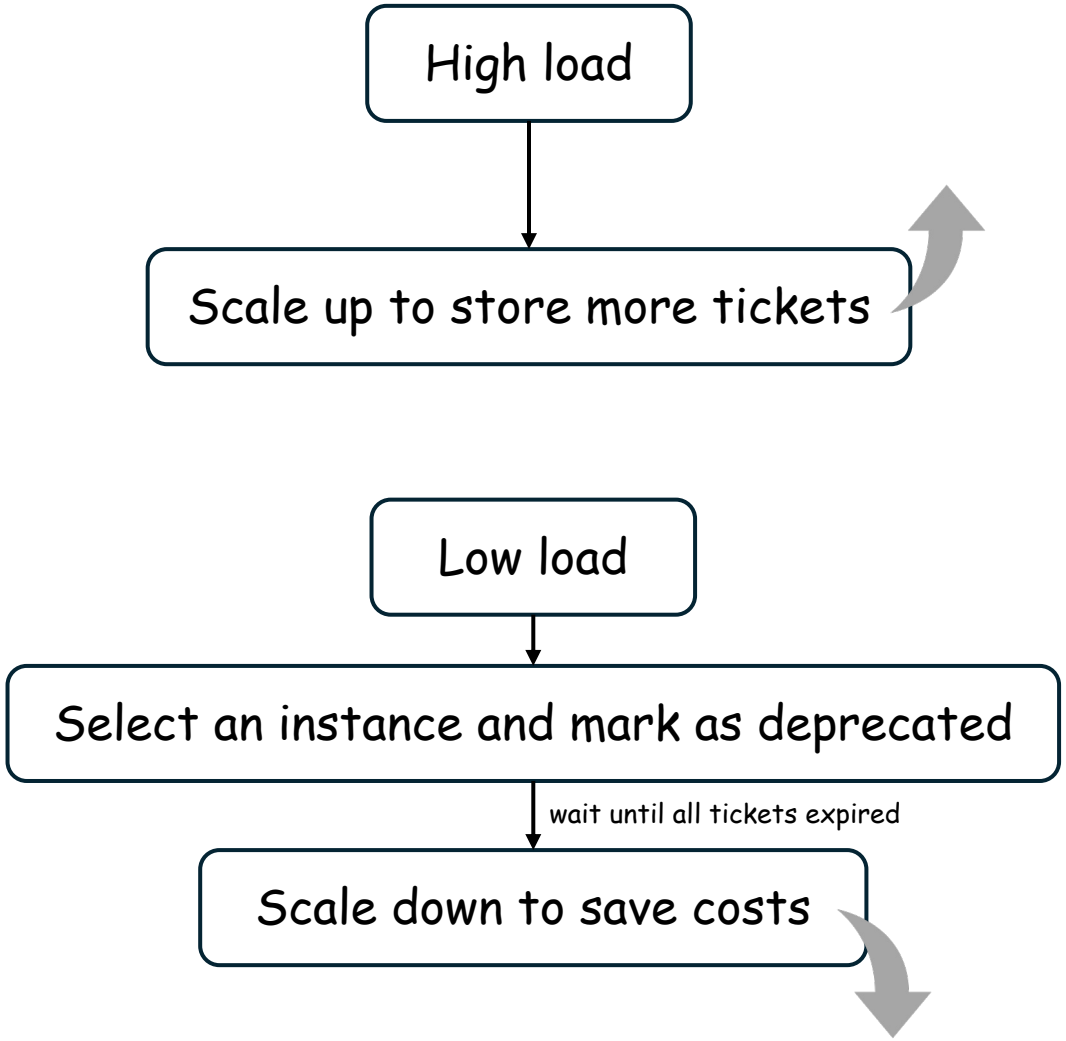
Simulator

High Performance

DS Integration

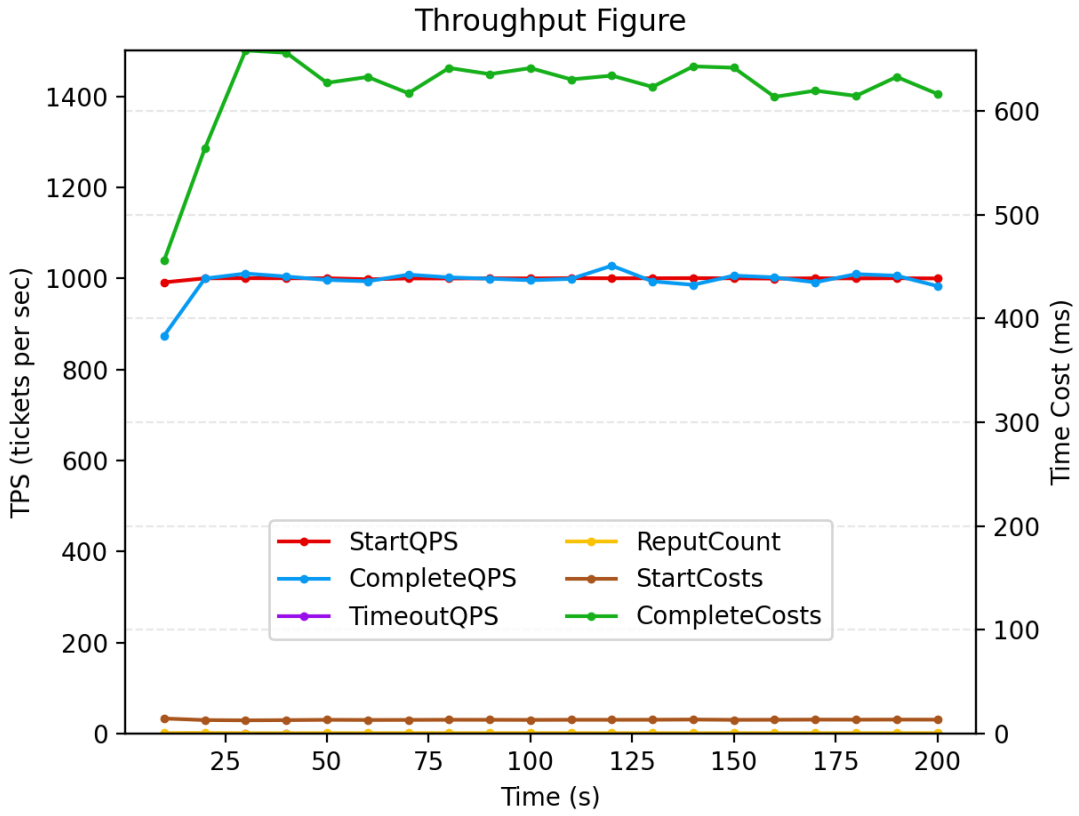
Ticket Pool AutoScale





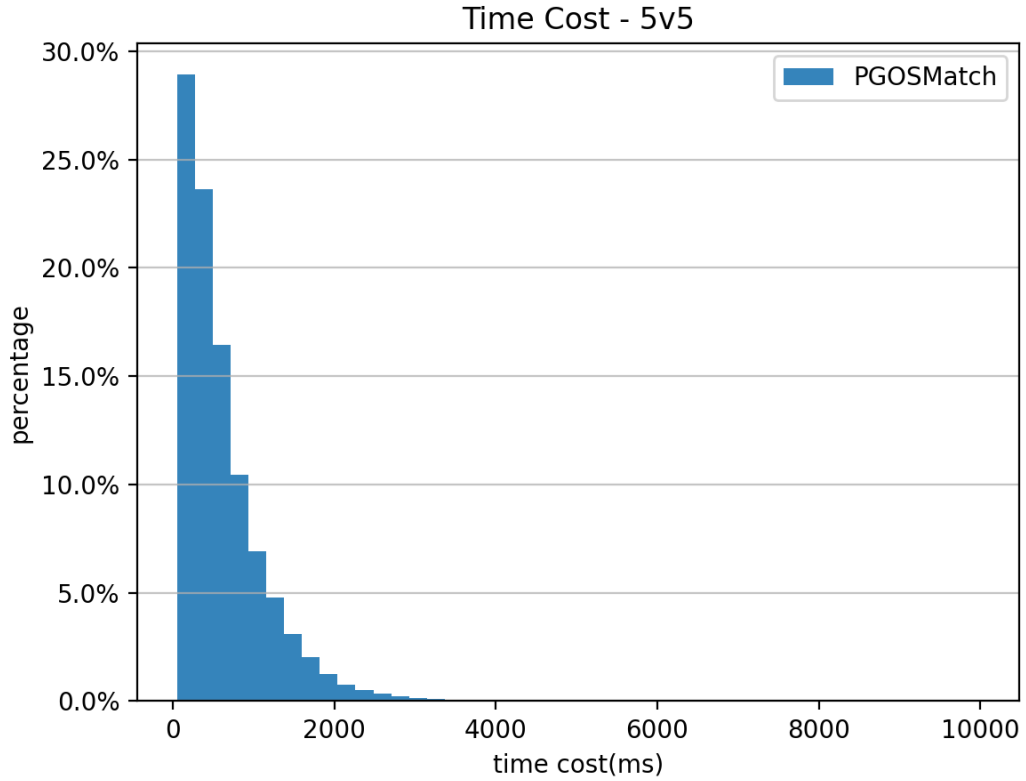
Matchmaking System Performance Evaluation

- *How to evaluate the throughput of a matchmaking system*
- *How to assess the user-side experience of matchmaking*



Matchmaking System Performance Evaluation

- *How to evaluate the throughput of a matchmaking system*
- *How to assess the user-side experience of matchmaking*



Topic List

- How to Build Fair Matches
- What If DSL Is Difficult to Debug
- How to Balance Match Quality and Queue Time
- Solving Performance Bottlenecks with Data Structures
- Improving Performance Limits Through Deployment Strategies
-  Real Case: When Clients Want Multi-Mode Matchmaking
- Emerging Trend: Engagement Oriented, Not Skill Based
- How to Extend MMR Calculation Mechanism with Virtual Server
- Consider Looter Shooters and Battle Royales

Multi Mode Matching





Casual Mode

-Team composition: 5v5, 4v4 或 3v3

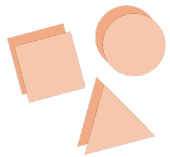
Ranked Mode

- Team composition: only 5v5

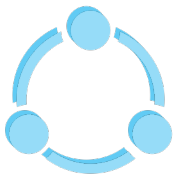
Multi Mode Matching



- Bitmap Number: ensure game mode selections by calculating bitand()



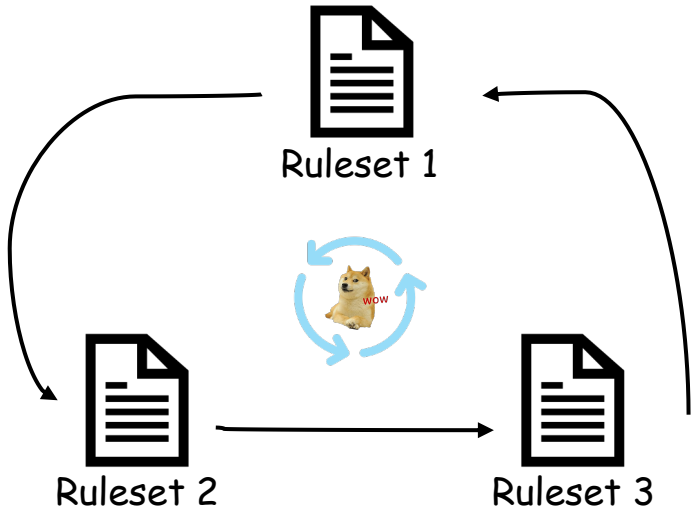
- Collection Rule: ensure game modes of players share >0 intersections



- Multi Rulesets: run multiple rulesets in parallel that share attributes

Multi Mode Matching

```
{
    "playerAttributes": [
        {
            "name": "mmr",
            "type": "number",
            "bitmap": false,
            "default": 1000,
            "partyAggregation": "each"
        },
        {
            "name": "mode",
            "type": "string",
            "bitmap": false,
            "default": "casual",
            "partyAggregation": "each"
        }
    ],
    "teams": [
        {
            "name": "red",
            "minPlayers": 3,
            "maxPlayers": 5,
        },
        {
            "name": "blue",
            "minPlayers": 3,
            "maxPlayers": 5,
        }
    ],
    "rules": [
        {
            "name": "fairness",
            "type": "distanceRule",
            "description": "Make the mmr of two teams close",
            "measurements": [
                "avg(teams[red].players.playerAttributes[mmr])"
            ],
            "referenceValue": "avg(teams[blue].players.playerAttributes[mmr])",
            "maxDistance": 100,
            "partyAggregation": "each"
        },
        {
            "name": "samemode",
            "type": "comparisonRule",
            "description": "Ensure players select same mode",
            "measurements": [
                "flatten(teams[*].players.playerAttributes[mode])"
            ],
            "referenceValue": "casual"
            "operation": "=",
            "partyAggregation": "each"
        }
    ]
}
```



The matchmaker iterates through the same set of players against three rulesets, and outputs a match result as long as any one rule is satisfied.

```
{
    "playerAttributes": [
        {
            "name": "mmr",
            "type": "number",
            "bitmap": false,
            "default": 1000,
            "partyAggregation": "each"
        },
        {
            "name": "mode",
            "type": "string",
            "bitmap": false,
            "default": "casual",
            "partyAggregation": "each"
        }
    ],
    "teams": [
        {
            "name": "red",
            "minPlayers": 5,
            "maxPlayers": 5,
        },
        {
            "name": "blue",
            "minPlayers": 5,
            "maxPlayers": 5,
        }
    ],
    "rules": [
        {
            "name": "fairness",
            "type": "distanceRule",
            "description": "Make the mmr of two teams close",
            "measurements": [
                "avg(teams[red].players.playerAttributes[mmr])"
            ],
            "referenceValue": "avg(teams[blue].players.playerAttributes[mmr])",
            "maxDistance": 100,
            "partyAggregation": "each"
        },
        {
            "name": "samemode",
            "type": "comparisonRule",
            "description": "Ensure players select same mode",
            "measurements": [
                "flatten(teams[*].players.playerAttributes[mode])"
            ],
            "referenceValue": "ranked"
            "operation": "=",
            "partyAggregation": "each"
        }
    ]
}
```

Topic List

- How to Build Fair Matches
- What If DSL Is Difficult to Debug
- How to Balance Match Quality and Queue Time
- Solving Performance Bottlenecks with Data Structures
- Improving Performance Limits Through Deployment Strategies
- Real Case: When Clients Want Multi-Mode Matchmaking
-  Emerging Trend: Engagement Oriented, Not Skill Based
- How to Extend MMR Calculation Mechanism with Virtual Server
- Consider Looter Shooters and Battle Royales

Engagement Oriented Matchmaking

EOMM: An Engagement Optimized Matchmaking Framework

ABSTRACT

Matchmaking connects multiple players to participate in online player-versus-player games. Current matchmaking systems depend on a single core strategy: create *fair games* at all times. These systems pair similarly skilled players on the assumption that a fair game is best player experience. We will demonstrate, however, that this intuitive assumption sometimes fails and that matchmaking based on fairness is not optimal for engagement.

In this paper, we propose an Engagement Optimized Matchmaking (EOMM) framework that maximizes overall player engagement. We prove that equal-skill based matchmaking is a special case of EOMM on a highly simplified assumption that rarely holds in reality. Our simulation on real data from a popular game made by Electronic Arts,Inc. (EA) supports our theoretical results, showing significant improvement in enhancing player engagement compared to existing matchmaking methods.

Keywords

matchmaking, player engagement, video games

1. INTRODUCTION

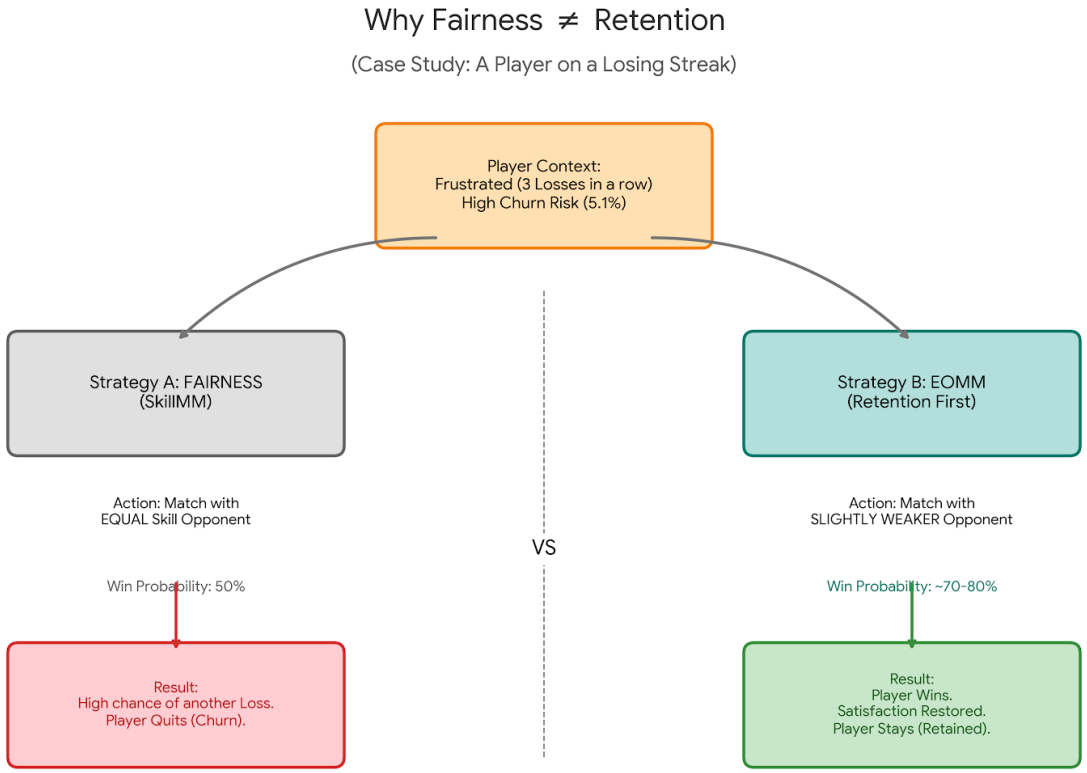
Player-versus-Player (PvP) is a mode of video game in which multiple players directly engage in competition or combat. PvP games, which cover many popular genres, such as multiplayer online battle arena (MOBA), first-person shooting (FPS), and e-Sports, have increased worldwide popularity in recent years. For example, *League of Legends*, one of the most played MOBA games, has 90 million summoner names registered, 27 million unique daily players and 7.5 million concurrent users [30, 41]. As data released by [39] shows, e-Sports is estimated to have 188 million viewership and 748 million dollar worth market in 2015 and the numbers are expected to grow continuously.

Matchmaking is the process that connects players to form PvP matches. In practice, a matchmaking system takes practical limitations, such as players' geo-location and network latency, into consideration. For example, cross-ocean pairing is not good for player experience. Beyond technical constraints, the strategy various matchmaking systems employ is *creating fair games*. This strategy relies on the assumption that matching closely skilled players tend to create competitive games which are desired by players [21]. In order to establish player skills, numerous models have been studied, such as Elo [16], Glicko [20] and TrueSkill [23].

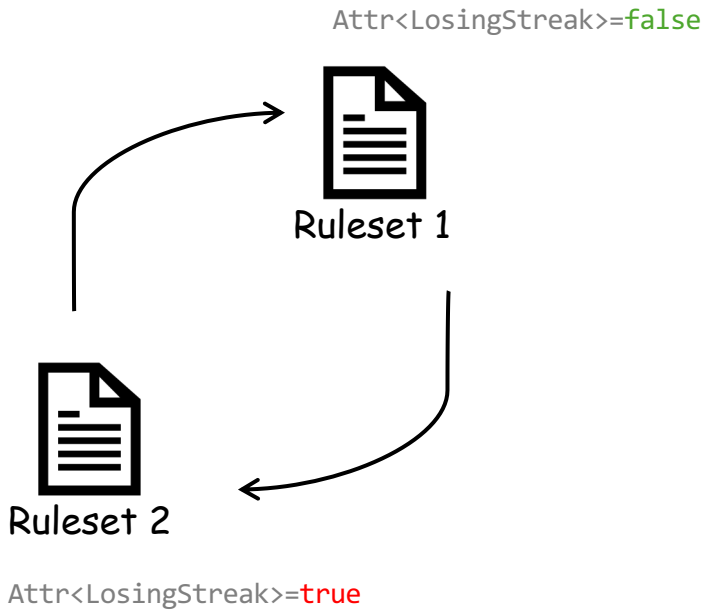
Are fairly matched games always beneficial for player experience? This fundamental, yet intuitive, assumption is worthy of deep investigation. We can challenge it with a few examples. Consider a cautious player who cares about protecting his rank among friends, and a risk taker who enjoys difficult matches. Pairing them with the similarly skilled opponents will affect these players very differently. Even for the same player, their expectation on the coming match when they just lost three games in a row can be very different from that when they recently performed well. In Table 1, we show an example that churn risks vary drastically upon players' recent match outcomes in a popular PvP game made by EA. These facts lead to two key insights: (1) the effectiveness of matchmaking needs to be measured quantitatively; and (2) matchmaking should depend on dynamic and individual *player states*.

In this paper, we propose a new matchmaking framework, Engagement Optimized Matchmaking (EOMM). By formulating matchmaking into an optimization problem, we pair players in order to maximize the overall player engagement, or equivalently, minimize the overall player disengagement. First, we measure a player's disengagement by their churn risk after each matchmaking decision. Here *churn* refers to no gameplay within a period of

*This work was done when Zhengxing Chen was an intern student at Electronic Arts,Inc.



Engagement Oriented Matchmaking



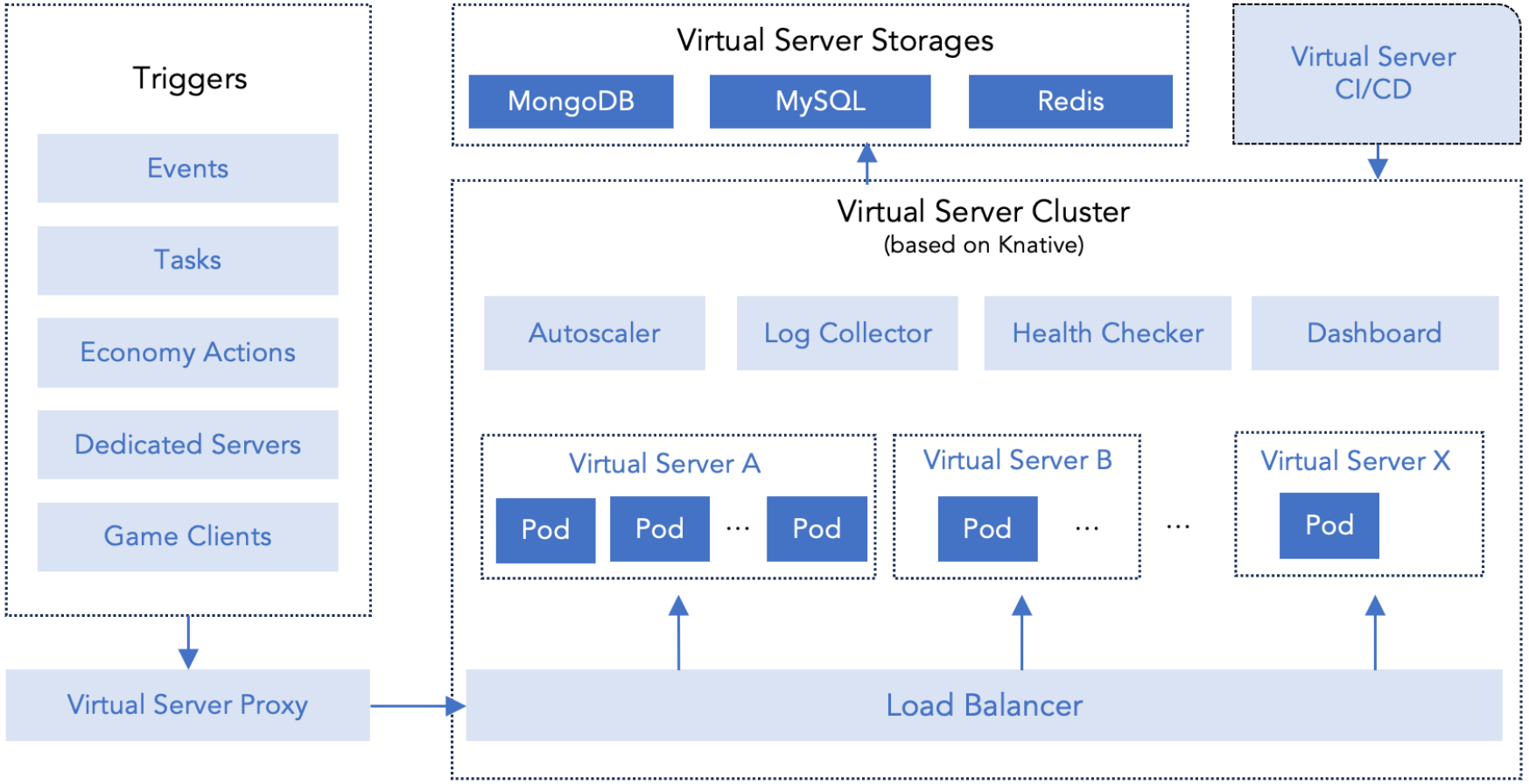
- Ruleset 1 for Skill Based Matching Strategy

- Ruleset 2 for Engagement Optimized Strategy

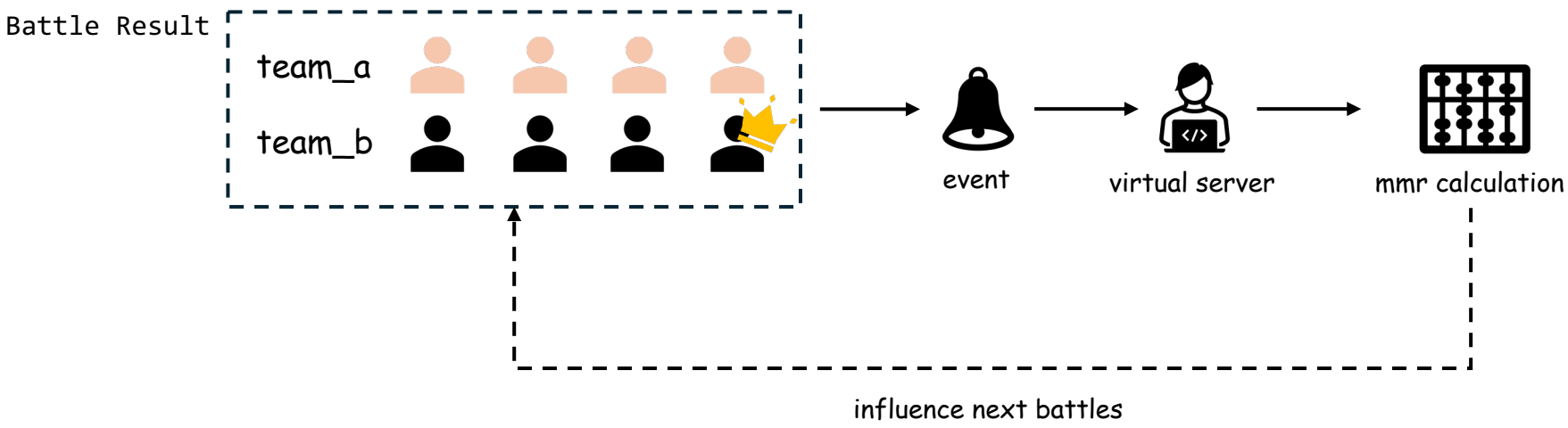
Topic List

- How to Build Fair Matches
- What If DSL Is Difficult to Debug
- How to Balance Match Quality and Queue Time
- Solving Performance Bottlenecks with Data Structures
- Improving Performance Limits Through Deployment Strategies
- Real Case: When Clients Want Multi-Mode Matchmaking
- Emerging Trend: Engagement Oriented, Not Skill Based
-  How to Extend MMR Calculation Mechanism with Virtual Server
- Consider Looter Shooters and Battle Royales

Extend MMR Mechanism via Virtual Server



Extend MMR Mechanism via Virtual Server



Topic List

- How to Build Fair Matches
- What If DSL Is Difficult to Debug
- How to Balance Match Quality and Queue Time
- Solving Performance Bottlenecks with Data Structures
- Improving Performance Limits Through Deployment Strategies
- Real Case: When Clients Want Multi-Mode Matchmaking
- Emerging Trend: Engagement Oriented, Not Skill Based
- How to Extend MMR Calculation Mechanism with Virtual Server
- Consider Looter Shooters and Battle Royales



Looter Shooters and Battle Royales

Aspect	*Battle Royale (PvPvP...)*	*Looter Shooter (PvE)*
Match Duration	**Medium**	**Long and Persistent**
Mid-Match Join	**Not Supported** All players must start simultaneously	**Supported & Critical** Backfill mechanism required
Skill Gap Tolerance	**Low to Medium Tolerance** Often uses tier-based matching (Bronze/Silver/Gold) rather than precise MMRComplete randomness frustrates new players	**High Tolerance** Skilled players can carry beginnersFocus more on "willingness to cooperate"MMR strict matching is less critical
Team Composition	**Large & Flexible**	**Small & Fixed** Typically 2-4
	Rule-based Matchmaker ✅	*World Matchmaker* ✅

Rule-based Matchmaker

StartMatchmaking

Pool

Waiting

Heuristic Search Algorithm

Fetch Players

According to Ruleset

x4 for each team

rules: gather two teams with different color,
but same color in one team, equal team size

Found a match!

team_a

team_b

DS Hosting

battle session

battle session

Fleet

Delayed Feedback

World Matchmaker

JoinWorld

Instant Response

Buckets

Exhibition-QA

session-1

Exhibition-Dev

session-2

session-3

Qualifying-QA

session-4

Qualifying-Dev

session-5

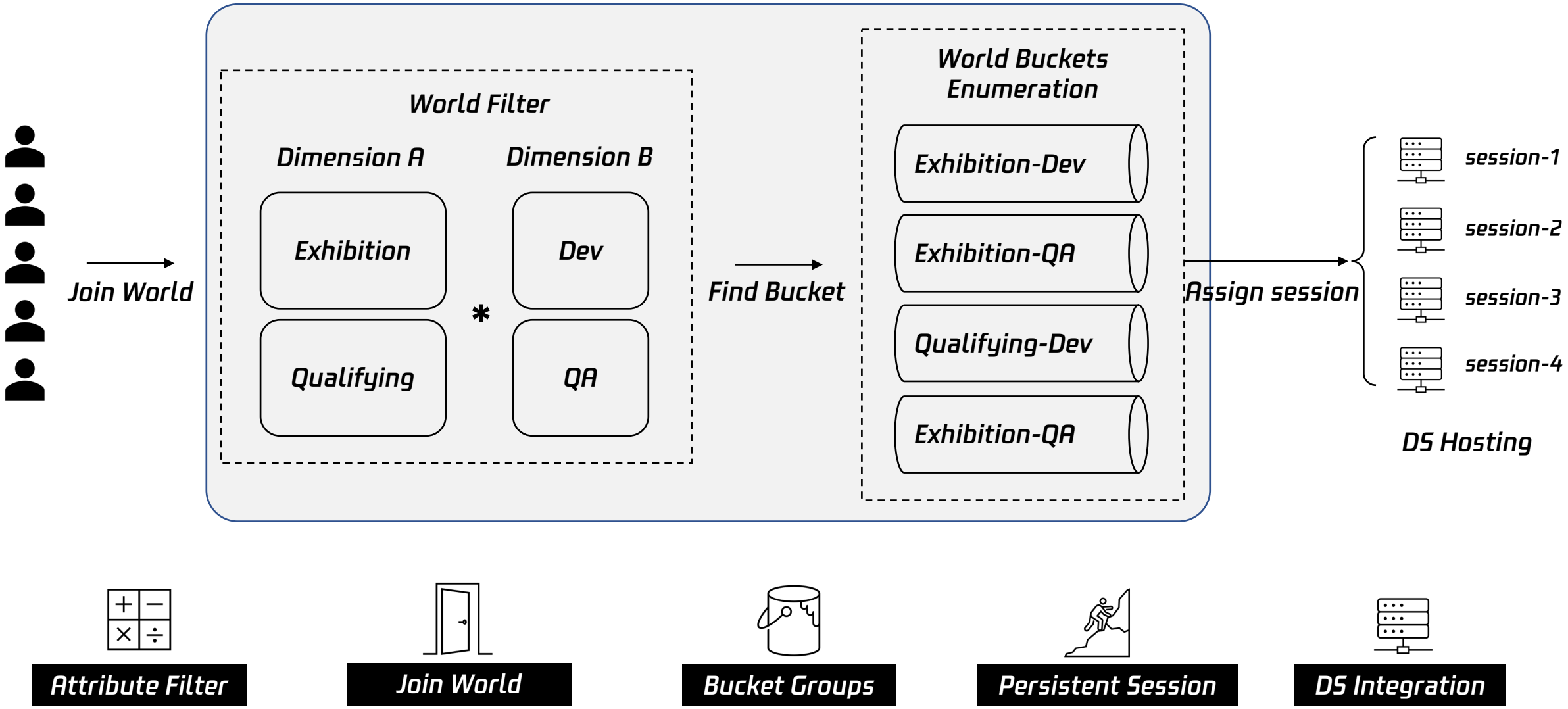
each bucket hold multiple persistent battle sessions,
which continuously wait for players to join

DS Hosting

session-1 session-2 session-3 session-4

Fleet

World Matchmaker



Attribute Filter

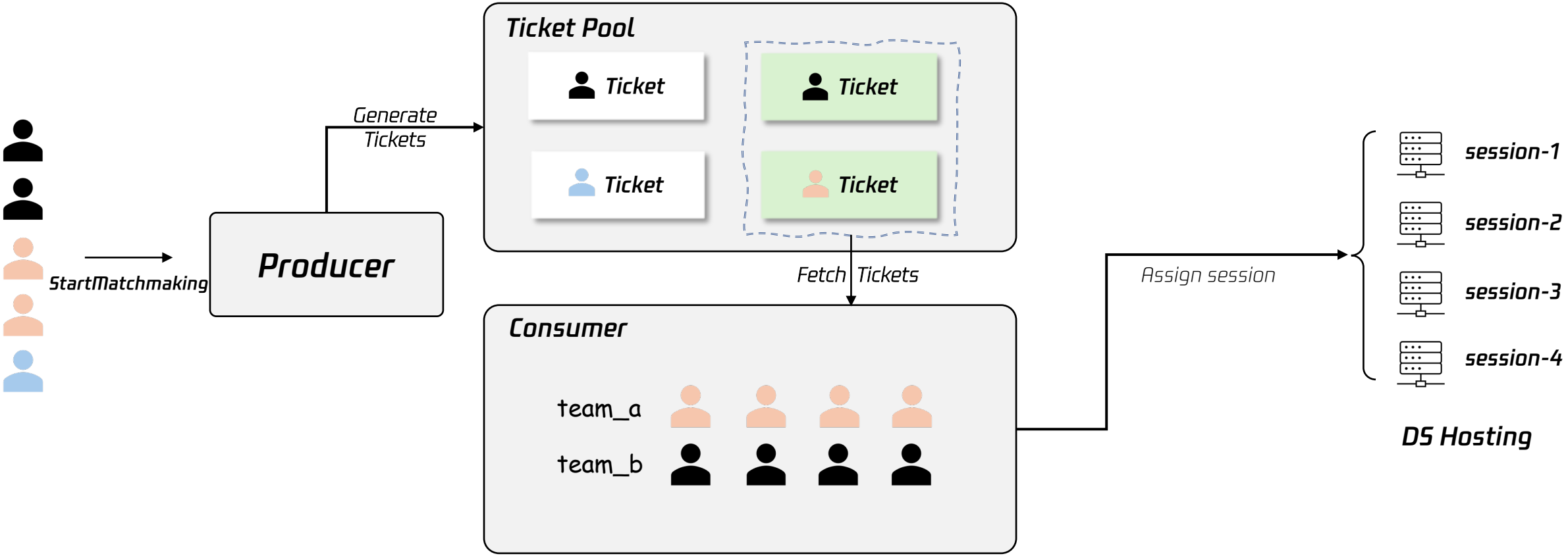
Join World

Bucket Groups

Persistent Session

DS Integration

Rule-based Matchmaker



Attribute Filter

Flexible Ruleset

Simulator

High Performance

DS Integration



Proposed the Multi-Mode Matching requirement, which PGOS has implemented.



Leverage Multi-Mode Matching to build the beginner-friendly match feature.



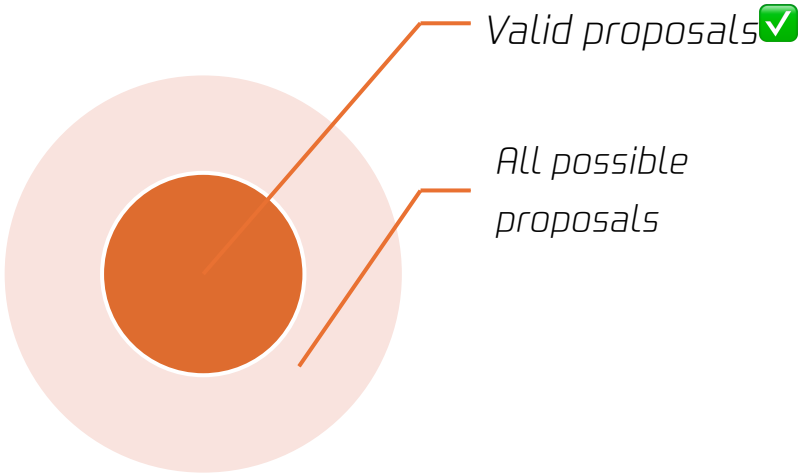
Efficiently test Rulesets using the Simulator.



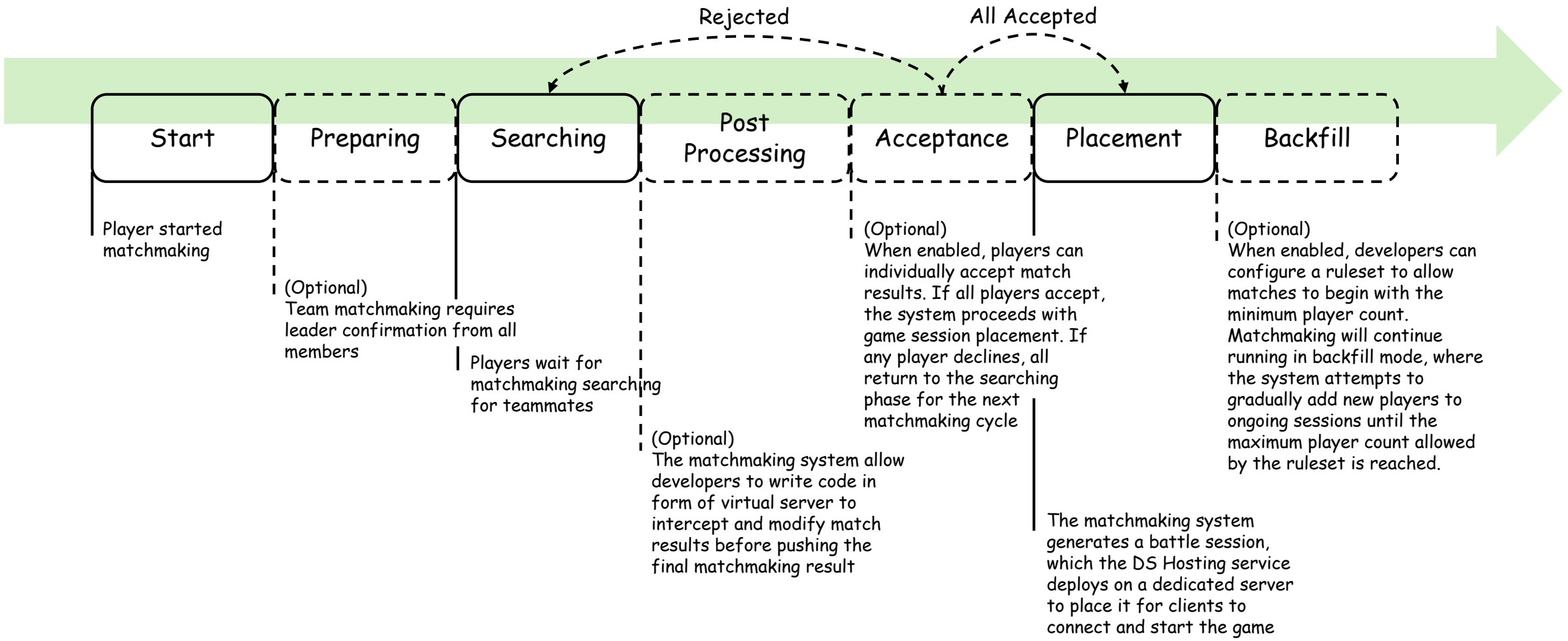
Appendix

Modeling the Matching Problem

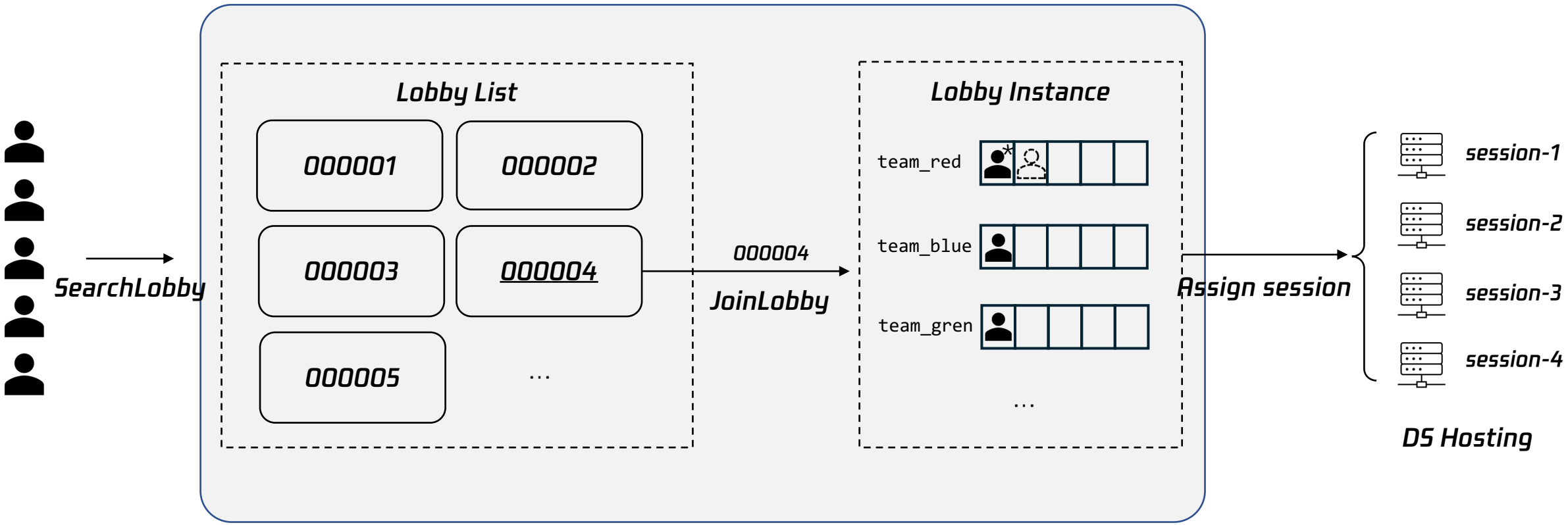
- *At its core, the matchmaking algorithm solves a search problem.*
- *It searches for a combination of p players that satisfy the rules from a pool of t fetched tickets.*
- *Core Parameters and Their Impact:*
  - *The values of t and p determine the worst-case time complexity of the matching search algorithm.*
  - *The strictness of the rules determines the proportion of valid proposals within the total search space.*



Full matchmaking workflow



Lobby



Manage Lobby

Join Lobby

Lobby List

Lobby Search

DS Integration

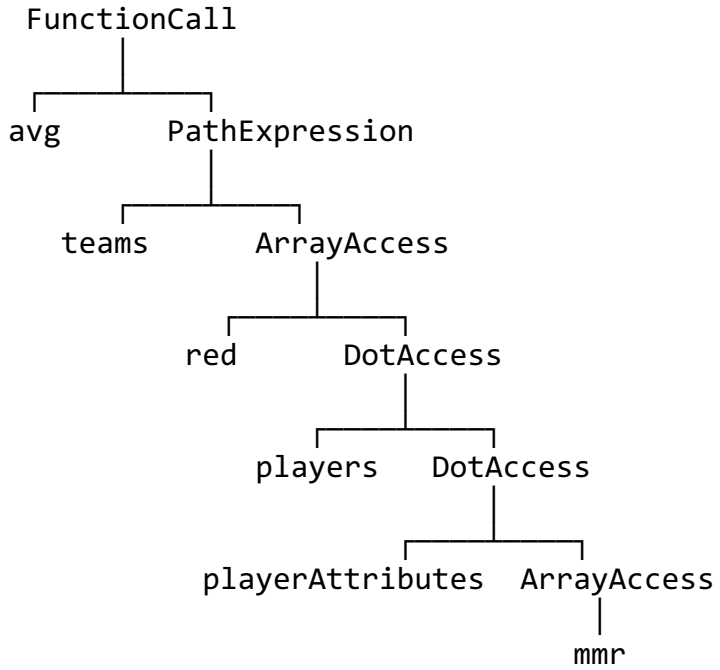
Break ruleset into parts: Expression

Expression: avg(teams[red].players.playerAttributes[mmr])

Lexical Analysis Phase:

Token Stream
FUNCTION(avg) → LPAREN → IDENTIFIER(teams) → LBRACKET → IDENTIFIER(red) → RBRACKET → DOT → IDENTIFIER(players) → DOT → IDENTIFIER(playerAttributes) → LBRACKET → IDENTIFIER(mmr) → RBRACKET → RPAREN

Syntax Analysis Phase - AST Construction:



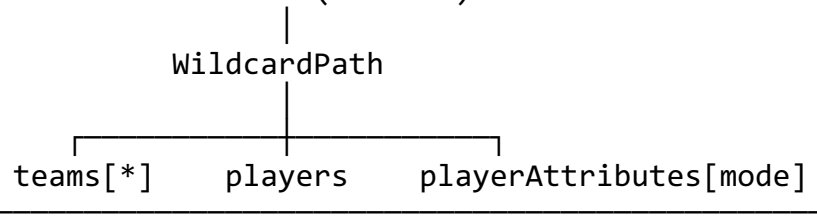
Expression: flatten(teams[*].players.playerAttributes[mode])

Parsing Process:

Step 1: Lexical Analysis

flatten	(	teams	[	*	]	.	players	...
FUNC	(	ID	[	*	]	.	ID	...

Step 2: Syntax Analysis - Wildcard Processing



Step 3: Semantic Analysis - Type Inference

```
teams[*] → Array<Team>
.players → Array<Array<Player>>
.playerAttributes[mode] → Array<Array<String>>
flatten() → Array<String>
```

Distance rule implementation

```
Rule Definition:
{
  "name": "fairness",
  "type": "distanceRule",
  "measurements": ["avg(teams[red].players.mmr)"],
  "referenceValue": "avg(teams[blue].players.mmr)",
  "maxDistance": 100
}
```

```
Calculation Process:

  Step 1: Calculate measurement value

    Red Team: [Player1(mmr:1200), Player2(mmr:1100),
              Player3(mmr:1300)]
    avg(teams[red].players.mmr) = (1200+1100+1300)/3
                                = 1200

  Step 2: Calculate reference value

    Blue Team: [Player4(mmr:1150), Player5(mmr:1250),
               Player6(mmr:1100)]
    avg(teams[blue].players.mmr) = (1150+1250+1100)/3
                                 = 1166.67

  Step 3: Distance calculation

    distance = |measurement - reference|
             = |1200 - 1166.67|
             = 33.33

    result = distance <= maxDistance
           = 33.33 <= 100
           = true (match successful)
```

PlayerID	MMR	GameMode	Assigned Team
Player1	1200	casual	red
Player2	1100	casual	red
Player3	1300	casual	red
Player4	1150	casual	blue
Player5	1250	casual	blue
Player6	1100	casual	blue

Comparison rule implementation

Rule Definition 1: Mode Consistency Check

```
{
  "name": "samemode",
  "type": "comparisonRule",
  "measurements": ["flatten(teams[*].players.mode)"],
  "operation": "="
}
```

Calculation Process:

Step 1: Expression evaluation

```
teams[*] → [red_team, blue_team]
teams[*].players → [[p1,p2,p3], [p4,p5,p6]]
teams[*].players.mode →[["casual","casual","casual"]
                        ["casual","casual","casual"]]
flatten(...) → ["casual","casual","casual",
                "casual","casual","casual"]
```

Step 2: Consistency check

```
operation = "="
Check if all elements are equal:
["casual"] = ["casual"] = ... = ["casual"] ?
result = true (all players have consistent mode)
```

Rule Definition 2: Team Size Balance

```
{
  "name": "equalteamsize",
  "type": "comparisonRule",
  "measurements": ["count(teams[red].players)"],
  "referenceValue": "count(teams[blue].players)",
  "operation": "="
}
```

Calculation Process:

```
measurement = count(teams[red].players) = 3
reference = count(teams[blue].players) = 3
result = (3 = 3) = true
```

Team	PlayerID	Game Mode
Red	Player1	casual
	Player2	casual
	Player3	casual
Blue	Player4	casual
	Player5	casual
	Player6	casual



pgos.levelinfinite.com

hannahrrli@tencent.com